

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

**Uma Arquitetura de Percepção e Mapeamento
para Ambientes Exógenos de Sistemas
Multi-Agentes Usando Visão Computacional**

Douglas da Costa Silva Bernardino

Prof. Orientador:

Carlos Eduardo Pantoja, D.Sc.

Diego Borda, D.Sc.

**Rio de Janeiro,
Janeiro de 2026**

**CEFET/RJ - CENTRO FEDERAL DE EDUCAÇÃO TECNOLÓGICA
CELSO SUCKOW DA FONSECA**

**Uma Arquitetura de Percepção e Mapeamento
para Ambientes Exógenos de Sistemas
Multi-Agentes Usando Visão Computacional**

Douglas da Costa Silva Bernardino

Projeto final apresentado em cumprimento às
normas do Departamento de Educação
Superior do Centro Federal de Educação
Tecnológica Celso Suckow da Fonseca,
CEFET/RJ, como parte dos requisitos para
obtenção do título de Bacharel em Sistemas de
Informação.

Prof. Orientador:
Carlos Eduardo Pantoja, D.Sc.
Diego Borda, D.Sc.

**Rio de Janeiro,
Janeiro de 2026**

DEDICATÓRIA

Dedico este trabalho à minha família que sempre esteve ao meu lado e me deu todo o suporte físico e emocional para que eu pudesse ter o privilégio de receber e partilhar conhecimento. Dedico também aos meus orientadores, que sempre esperaram mais de mim e me ensinaram, através do exemplo, a buscar uma versão própria cada vez melhor.

AGRADECIMENTOS

Em primeiro lugar, agradeço a Deus por tudo em minha vida. Também, um grande agradecimento a todos que contribuíram direta e indiretamente durante todo o meu progresso nesta jornada, principalmente aos meus pais que sempre fizeram o possível e impossível para que eu pudesse estar aqui hoje, agradeço minha esposa por todo apoio e parceria. Um agradecimento especial também aos meus orientadores Carlos Pantoja e Diego Borda por todo ensinamento, desafio e confiança.

RESUMO

No cenário atual da robótica e Inteligência Artificial Distribuída, a percepção precisa do ambiente é um dos maiores desafios para a autonomia de agentes. Enquanto sistemas tradicionais dependem de sensores embarcados (endógenos), muitas vezes limitados por custo e processamento, a percepção exógena surge como uma alternativa robusta para coordenação global. Este trabalho apresenta uma proposta de arquitetura para o mapeamento de ambientes exógenos em Sistemas Multi-Agentes (SMA) utilizando Visão Computacional. A solução utiliza o algoritmo YOLO para detecção de objetos e o framework JaCaMo para o raciocínio dos agentes. Diferente de abordagens puramente teóricas, este estudo foca na integração prática entre a camada de percepção visual e a tomada de decisão BDI. Os resultados demonstram a viabilidade do mapeamento em tempo real com seis protótipos, apesar das limitações de latência identificadas, contribuindo para o desenvolvimento de sistemas de monitoramento e logística inteligente.

Palavras-chaves: Sistemas Multiagente. JaCaMo. Robótica Autônoma. Visão Computacional.

ABSTRACT

In the contemporary scenario of robotics and Distributed Artificial Intelligence, accurate environmental perception is the foundation for agent autonomy. While traditional approaches rely on endogenous (internal) sensors, which frequently increase hardware cost and complexity, this work proposes the use of exogenous perception as a strategic alternative. This study presents an architecture for environmental mapping using Computer Vision integrated with Multi-Agent Systems (MAS). The proposed solution utilizes the YOLO algorithm for object detection and the JaCaMo framework for agent reasoning. Unlike isolated vision systems, this research focuses on creating a data bridge that transforms video frames into beliefs for the BDI model. The results demonstrate that, despite an identified latency of 550ms, the architecture is viable for the global mapping of multiple agents, filling a technical integration gap between deep computer vision and agent logic.

Keywords:

Sumário

1	Introdução	1
1.1	Motivação	3
1.2	Objetivo	3
1.3	Contribuições	3
1.4	Estrutura do trabalho	5
2	Conceitos Básicos	6
2.1	Agentes e Sistemas Multi-Agentes (SMA)	6
2.1.1	Framework JaCaMo	13
2.2	Visão Computacional	13
2.3	Algoritmos de Machine Learning e Deep Learning	14
2.4	Tecnologias Utilizadas	14
3	Identificação e Coleta de Objetos no Ambiente Exógeno	17
3.1	Metodologia	17
3.1.1	Arquitetura do Sistema	19
3.2	Identificação e Interação com o Objeto	21
3.2.1	Monitoramento com Câmera	21
3.2.2	Desafios e Robustez da Percepção Visual	23
3.2.3	Algoritmos de Detecção	24
3.2.4	Envio de Dados para o Agente	26
3.3	Modelo Conceitual	27
3.3.1	Definição de Crenças, Intenções, Desejos e Planos	28
3.3.2	Implementação do Agente Carro	31
4	Implementação	35
4.1	Limitações Atuais	35
4.2	Validação e Detalhamento da Camada de Percepção Visual: A Contribuição Efetiva	36
4.2.1	Detalhamento do Processo de Detecção e Localização de Objetos	36
4.2.2	Localização e Discretização de Coordenadas	41

4.2.3	Validação da API de Comunicação (Middleware)	41
4.3	Validação Experimental do Mapeamento de Agentes	43
5	Conclusão	51
5.1	Considerações Finais	51
5.2	Perspectivas para Pesquisa Futura	52
	Referências Bibliográficas	53

Lista de Figuras

FIGURA 1:	Modelo Conceitual	18
FIGURA 2:	Modelo Pré Treinado	24
FIGURA 3:	DataSet com Tensorflow para Classificação	37
FIGURA 4:	Modelo de Classificação com Tensorflow	37
FIGURA 5:	Gerenciamento e Anotação Manual das Imagens	38
FIGURA 6:	Divisão de Conjunto de Dados	39
FIGURA 7:	Detecção de Objeto	40
FIGURA 8:	Discretização de Coordenadas	41
FIGURA 9:	Estrutura de Dados Retornada pela API de Comunicação	42
FIGURA 10:	Teste de Percepção - Agente Decocco	44
FIGURA 11:	Teste de Percepção - Agente Golarmi	45
FIGURA 12:	Teste de Percepção - Agente KombiCeptor	46
FIGURA 13:	Teste de Percepção - Agente KombusPrime	47
FIGURA 14:	Teste de Percepção - Agente Margheriti	48
FIGURA 15:	Teste de Percepção - Agente Samu	49

Lista de Tabelas

TABELA 1:	Métricas de Desempenho da Camada de Percepção Visual	40
TABELA 2:	Resultados da Extração de Percepção - Agente Decocco	44
TABELA 3:	Resultados da Extração de Percepção - Agente Gorlami	45
TABELA 4:	Resultados da Extração de Percepção - Agente KombiCeptor	46
TABELA 5:	Resultados da Extração de Percepção - Agente KombusPrime	47
TABELA 6:	Resultados da Extração de Percepção - Agente Margheriti	48
TABELA 7:	Resultados da Extração de Percepção - Agente Samu	49

LISTA DE ABREVIACES

Capítulo 1

Introdução

O uso de sistemas autônomos em ambientes reais, como logística inteligente e monitoramento urbano, demanda uma capacidade de percepção que vá além de sensores internos. A distinção entre ambientes endógenos — onde o sistema possui controle total e percepção completa — e ambientes exógenos — caracterizados pela imprevisibilidade e dinamismo — é fundamental para o design de agentes inteligentes.

Este trabalho apresenta uma proposta de arquitetura para o mapeamento de ambientes exógenos em Sistemas Multi-Agentes (SMA) utilizando Visão Computacional. O foco desta pesquisa não reside na definição teórica da arquitetura de agentes, mas na implementação prática de uma "ponte de dados" que permita a agentes autônomos interpretar um ambiente físico externo em tempo real. A solução proposta integra a detecção de objetos via algoritmo YOLO ao framework JaCaMo, permitindo que a camada de visão computacional alimente o modelo cognitivo BDI do agente.

A natureza do ambiente em que um agente opera pode definir o seu design e complexidade. Wooldridge [2009] classifica os ambientes com base em propriedades como acessibilidade (se o agente tem percepção completa ou incompleta do estado), determinismo (se o resultado das ações é previsível), episodicidade (se as ações presentes afetam o futuro), dinamismo (se o ambiente muda independentemente do agente) e discretização (se os estados e ações são finitos ou contínuos). Ambientes mais dinâmicos, inacessíveis e não determinísticos exigem agentes mais sofisticados, capazes de lidar com a incerteza e a mudança em tempo real para tomar decisões eficazes.

Há dois ambientes distintos em que os SMA Embarcados podem atuar, são eles:

- **Ambientes Endógenos:** É um ambiente que está completamente descrito e acessível dentro do sistema. Isso significa que o agente ou sistema tem controle total sobre as regras e variáveis que o compõem. As percepções do agente nesse ambiente são finitas e previsíveis, uma vez que o ambiente é projetado de forma controlada. Exemplos incluem jogos de tabuleiro como xadrez, onde todas as regras e estados possíveis estão definidos

e são conhecidos pelos agentes.

- **Ambientes Exógenos:** É um ambiente que existe fora do controle direto do sistema ou agente. Esse tipo de ambiente é externo e geralmente mais difícil de lidar, pois envolve interações com um mundo real ou simulado que é dinâmico, contínuo e imprevisível. As percepções tendem ao infinito porque o agente não tem uma visão completa ou detalhada de tudo o que acontece no ambiente. Exemplos incluem sistemas que interagem com o mundo físico, como robôs em ambientes urbanos, onde as variáveis do ambiente (como clima, trânsito ou comportamento humano) mudam constantemente de forma imprevisível.

Essas distinções fundamentais na atuação de SMAs embutidos são corroboradas pelas classificações de ambiente propostas por Wooldridge [2009]. As características de um Ambiente Endógeno (completo, acessível, com regras e variáveis sob controle, e percepções finitas e previsíveis) correspondem diretamente aos ambientes que ele classifica como acessíveis, determinísticos, discretos e frequentemente estáticos. Por outro lado, as propriedades de um Ambiente Exógeno (fora de controle direto, dinâmico, contínuo, imprevisível, com percepções incompletas ou infinitas) refletem os ambientes que Wooldridge [2009] descreve como inacessíveis, não determinísticos, contínuos e dinâmicos. Essa correlação demonstra a importância de considerar a natureza do ambiente no desenvolvimento e na aplicação de SMA.

Nesse caso os agentes atuarão em um ambiente exógeno, tendo suas percepções potencialmente infinitas, já que o ambiente pode mudar de formas imprevisíveis. Devido a essa complexidade, lidar com ambientes exógenos requer métodos avançados de adaptação e aprendizado, uma vez que o agente não pode prever todas as variáveis externas, a visão computacional é uma solução para manter os Agentes e o SMA Embarcado informados sobre toda e qualquer variação do ambiente.

A relevância deste projeto reside na demonstração prática da arquitetura proposta. Contudo, em virtude do escopo e do prazo deste trabalho de conclusão de curso, a entrega final se concentra na validação da Camada de Percepção Visual e na implementação da API de Comunicação. O objetivo é apresentar um sistema robusto capaz de identificar e localizar objetos em um ambiente exógeno, provendo a base de dados necessária para a camada de decisão dos agentes, que, por sua vez, é detalhada como Trabalho Futuro.

1.1 Motivação

Atualmente, a maioria das soluções em robótica autônoma oscila entre dois extremos: sistemas centralizados, como o futebol de robôs, que operam em ambientes altamente controlados (endógenos); e sistemas baseados puramente em sensores de localização (GPS/GSM), que, embora eficazes para rastreamento, carecem da riqueza semântica necessária para identificar tipos de objetos ou anomalias visuais. O desafio reside em integrar a autonomia distribuída de um SMA com a complexidade de um ambiente não estruturado. Este trabalho justifica-se pela necessidade de um sistema que permita a agentes cognitivos atuar em cenários onde a percepção visual é essencial para a tomada de decisão proativa, superando as limitações dos sensores de hardware limitados pelo custo e pela complexidade.

1.2 Objetivo

O objetivo deste trabalho é propor e validar uma arquitetura híbrida de percepção exógena, capaz de identificar protótipos de veículos terrestres e objetos específicos, convertendo percepções visuais em crenças processáveis pelo framework JaCaMo. O escopo final concentra-se na validação da camada de percepção visual e da API de comunicação.

1.3 Contribuições

Este trabalho visa contribuir com a área de robótica autônoma e sistemas multiagente de diversas maneiras significativas:

1. Modelo de Visão Computacional

- Desenvolvimento de um Modelo de Visão Computacional Customizado: Foco no treinamento de um modelo de aprendizado de máquina capaz de detectar e localizar com precisão o protótipo de veículo terrestre autônomo e o objeto específico (bola de plástico) a partir de uma câmera externa, garantindo a extração das coordenadas em tempo real.

2. Integração de Visão Computacional com o Sistema Multiagente

- Proposição de uma Arquitetura de Integração entre Visão Computacional e SMA: Criação de uma arquitetura que estabelece uma comunicação eficiente e robusta

entre o módulo de visão computacional (em Python) e o sistema multiagente embarcado (em JaCaMo), permitindo a troca contínua e em tempo real de informações cruciais, como a localização dos objetos detectados.

3. Mapeamento do Ambiente e Localização de Objetos

- Desenvolvimento de um Sistema de Mapeamento e Localização de Objetos no Ambiente Físico: Implementação de um método para converter as coordenadas de pixel obtidas pela câmera em um mapeamento preciso no plano físico (cartesianas) do ambiente. Essa capacidade de localização exata do veículo e do objeto é fundamental para o raciocínio dos agentes e a navegação autônoma.

4. Deliberação e Coordenação dos Agentes

- Implementação de Agentes Cognitivos para Deliberação e Tomada de Decisão: Desenvolvimento de agentes inteligentes no framework JaCaMo, dotados da capacidade de receber e interpretar as percepções visuais do ambiente, deliberar sobre as coordenadas do veículo e do objeto, e executar ações autônomas de navegação e interação, visando o objetivo final de alcançar o objeto.

5. Validação Prática

- Validação Experimental da Solução Integrada: Realização de testes práticos em um ambiente controlado, utilizando protótipos de veículos terrestres e o objeto (bola), para demonstrar a funcionalidade e a eficácia da integração entre visão computacional e o sistema multiagente na execução de comandos de navegação e interação baseados nas percepções em tempo real.

6. Automação e Escalabilidade

- Proposição de uma Solução Escalável e de Baixo Custo: Apresentação de uma arquitetura que não só automatiza o processo de navegação e interação, mas que também é inerentemente escalável (facilitando a adição de mais agentes ou cenários) e de baixo custo de implementação, tornando-a acessível para futuras pesquisas e aplicações.

7. Contribuições Tecnológicas

- Sistematização de uma Abordagem Distribuída para Ambientes Exógenos: Consolidação de uma metodologia que integra visão computacional e SMA embarcados para a interação em ambientes exógenos, explorando a deliberação distribuída e as capacidades cognitivas Belief, Desire and Intention(BDI) dos agentes para lidar com a complexidade e imprevisibilidade do mundo real.

1.4 Estrutura do trabalho

O trabalho está organizado da seguinte forma: na Seção 2, são apresentados os conceitos básicos necessários para a compreensão do modelo proposto. Na Seção 3, é detalhada a arquitetura do projeto e os direcionamentos tomados para sua implementação. Na Seção 4, são apresentadas as limitações atuais do projeto, sua implementação e é dada ênfase em seus testes e validações. Na Seção 5, são apresentadas as considerações finais, discutindo os resultados obtidos e possíveis direções para trabalhos futuros. Esta proposta de modelo visa integrar a tecnologia autônoma e o modelo cognitivo BDI, proporcionando uma solução eficaz para o desafio de navegação e interação de agentes em ambientes exógenos. Para validar essa proposta, optou-se por utilizar um modelo terrestre, no qual um carro autônomo interage com um objeto específico (bola), facilitando a montagem e a realização dos testes práticos.

Capítulo 2

Conceitos Básicos

Para a compreensão aprofundada da arquitetura proposta neste trabalho, bem como os desafios e as soluções empregadas na interação de sistemas autônomos com ambientes dinâmicos, faz-se necessário apresentar os fundamentos teóricos que embasam o desenvolvimento. Esta seção dedica-se a introduzir os conceitos essenciais de SMA, destacando sua relevância na criação de inteligência distribuída e colaborativa. Em seguida, exploramos os princípios da Visão Computacional, fundamental para a percepção do ambiente em tempo real, e as bases dos Algoritmos de ML e DL, que permitem aos sistemas aprender e identificar padrões complexos. Por fim, detalharemos as Tecnologias Utilizadas na implementação, consolidando o conhecimento necessário para o entendimento das próximas etapas do trabalho.

2.1 Agentes e Sistemas Multi-Agentes (SMA)

Os SMA formam uma subárea da Inteligência Artificial Distribuída (IAD), com foco no estudo de agentes autônomos operando em um universo compartilhado. Nesses sistemas, o termo **autônomo** designa que os agentes possuem uma existência própria e independente. Cada agente detém um conjunto de capacidades comportamentais que definem sua competência, um conjunto de objetivos e a autonomia necessária para empregar essas capacidades a fim de alcançar suas metas.

Um agente é uma entidade de software que realiza ações em um ambiente de forma autônoma, visando alcançar objetivos específicos. Ele é capaz de perceber o ambiente por meio de sensores e agir sobre ele com atuadores, com o propósito de maximizar algum critério de desempenho [Russell and Norvig, 2010]. Sendo assim o comportamento autônomo lhe permite decidir suas próprias ações [Alvares and Sichman, 1997]. A decisão de qual ação realizar é determinada pelo agente, considerando as mudanças no ambiente em que atua e o desejo de alcançar seus objetivos. A ideia principal em um sistema multiagente é que um comportamento global inteligente pode ser alcançado a partir do comportamento individual dos agentes. Em um SMA, não é necessário que cada agente seja individualmente inteligente para alcançar um

comportamento global inteligente; eles podem competir, cooperar e interagir para alcançar tal objetivo. As principais características de agentes de um SMA segundo Wooldridge [2009] são:

- **Autonomia:** Cada agente em um SMA opera de maneira independente, com sua própria percepção e habilidades. A autonomia permite que os agentes decidam suas ações com base em suas observações e objetivos, sem dependerem diretamente de outros agentes.
- **Interação:** Agentes interagem entre si e com o ambiente, trocando informações e coordenando atividades. Essas interações podem ser cooperativas, competitivas ou neutras, dependendo dos objetivos dos agentes.
- **Proatividade:** Que os leva a tomar a iniciativa além de simplesmente reagir ao ambiente, possibilitando a interação entre agentes para cooperar na busca de objetivos.
- **Cooperação e Coordenação:** Em muitos casos, os agentes precisam cooperar para alcançar objetivos comuns ou coordenar suas ações para evitar conflitos. Isso pode envolver comunicação direta, negociação e mecanismos de consenso.
- **Comportamento Global:** O comportamento global do sistema é emergente, ou seja, resultado da interação dos comportamentos dos indivíduos dos agentes. Mesmo que os agentes não tenham inteligência global, o sistema pode exibir inteligência global por meio de suas interações e cooperação.

Nem todos os problemas podem ser resolvidos de forma centralizada. Um exemplo é o controle das linhas de metrô, onde cada estação possui suas próprias regras e particularidades. Por isso, é necessário um controle distribuído, no qual as informações de cada estação são integradas para coordenar o sistema como um todo. Essa distribuição de tarefas em uma IA é conhecida como IAD. Nesse contexto, os SMA se destacam pela capacidade de distribuir tarefas entre agentes autônomos, proativos e comunicativos. Essas características permitem que os agentes colaborem e se adaptem ao ambiente, interagindo de forma eficiente para resolver problemas em sistemas dinâmicos e descentralizados.

Para entender a comunicação e negociação entre agentes em um SMA, é essencial explorar os padrões de interação definidos pela Foundation for Intelligent Physical Agents (FIPA). Dentre os protocolos mais utilizados, destacam-se:

- **FIPA Request:** Este protocolo define como um agente pode solicitar que outro realize uma ação específica. O agente emissor envia uma mensagem com a ação desejada, e o

receptor responde confirmando ou rejeitando a solicitação, podendo também fornecer justificativas. É amplamente utilizado em sistemas onde a delegação de tarefas é frequente, como em ambientes colaborativos [Foundation for Intelligent Physical Agents (FIPA), 2000].

- **FIPA Contract Net:** Um protocolo de negociação baseado em um modelo de licitação. O agente iniciador (licitador) anuncia uma tarefa a ser executada e os agentes participantes (licenciados) apresentam propostas para realizá-la. O iniciador avalia as propostas recebidas e seleciona a mais adequada. Esse protocolo é particularmente útil em cenários onde tarefas devem ser distribuídas de forma eficiente entre múltiplos agentes, como na alocação de recursos ou agendamento de processos [Foundation for Intelligent Physical Agents (FIPA), 2000].
- **FIPA Subscribe:** Este protocolo permite que um agente solicite a um outro notificações sobre mudanças em um determinado evento ou condição. O receptor da solicitação monitora a condição e informa o solicitante sempre que ocorrer uma alteração. É amplamente aplicado em sistemas dinâmicos, onde o acompanhamento em tempo real de mudanças no ambiente é necessário [Foundation for Intelligent Physical Agents (FIPA), 2000].

Além dos protocolos FIPA, o Knowledge Query and Manipulation Language (KQML) desempenha um papel em SMA, incluindo os desenvolvidos com o framework Jason. O KQML é uma linguagem de comunicação de agentes que permite a troca de informações e comandos de maneira estruturada e semântica. Ele suporta diversos atos de fala, como perguntas, respostas, pedidos e anúncios, facilitando tanto a comunicação quanto o raciocínio entre os agentes.

No Jason, os atos de fala do KQML são mapeados diretamente para planos de comunicação dos agentes, permitindo uma implementação clara e eficiente de interações. Por exemplo, mensagens do tipo *tell* ou *ask* são comuns no contexto de SMA baseados em Jason, refletindo as necessidades de informar ou questionar outros agentes sobre o estado do ambiente ou objetivos compartilhados.

Combinando os protocolos FIPA e as capacidades semânticas do KQML, sistemas multiagentes podem alcançar uma comunicação rica e flexível, essencial para atender às demandas de ambientes distribuídos e dinâmicos.

Entre os diversos seguimentos da IA, alguns são dedicados ao estudo de arquiteturas de agentes deliberativos baseadas em estados mentais. Entre estes estados mentais estão crenças,

desejos e intenções, que ajudam a definir como os agentes percebem o mundo, o que desejam alcançar e quais planos seguem para atingir seus objetivos. As arquiteturas que utilizam esta tríade de estados são denominadas como arquiteturas BDI [Rao and Georgeff, 1995].

Nem todos os agentes inteligentes adotam o modelo BDI. De maneira geral, os agentes podem ser classificados em duas categorias principais: agentes reativos e agentes cognitivos. Agentes reativos baseiam-se em estímulo-resposta, ou seja, seu comportamento é diretamente desencadeado por mudanças no ambiente, sem envolver processos internos complexos. Por outro lado, agentes cognitivos possuem estruturas internas que permitem raciocínio e deliberação, sendo capazes de planejar, tomar decisões e agir de forma mais sofisticada.

Dentro da classe dos agentes cognitivos, diversos modelos podem ser utilizados para estruturar suas capacidades internas, e o modelo BDI (Belief-Desire-Intention) é um dos mais conhecidos e amplamente adotados.

O modelo BDI é uma arquitetura que organiza o comportamento do agente em torno de três componentes principais: crenças (beliefs), que representam as informações que o agente considera verdadeiras sobre o mundo; desejos (desires), que são os objetivos que ele pretende alcançar; e intenções (intentions), que correspondem aos planos que o agente escolhe executar para atingir seus objetivos.

O principal propósito do modelo BDI é capturar, de maneira estruturada, os processos de deliberação e execução que guiam o comportamento de agentes autônomos. As crenças fornecem o contexto necessário para a tomada de decisão; os desejos representam os estados futuros desejados; e as intenções especificam os compromissos do agente em relação aos planos escolhidos. Essa organização permite que os agentes adaptem seu comportamento ao ambiente dinâmico e priorizem ações de acordo com seus objetivos.

Uma característica importante do modelo BDI é sua separação entre a deliberação (seleção de planos a partir de um conjunto de possibilidades) e a execução (implementação das ações contidas nos planos ativos). Contudo, a criação inicial dos planos, ou seja, o processo de planejamento detalhado, geralmente está fora do escopo do modelo BDI e depende do projetista ou do programador do sistema.

Implementações práticas do modelo, como a linguagem AgentSpeak ou plataformas como Jason, traduzem os conceitos do modelo BDI em componentes concretos de software, facilitando a programação de agentes que precisam atuar de maneira racional e eficiente em ambientes dinâmicos e incertos [Bordini et al., 2007]. Para melhor compreender como esses agentes

operam, é fundamental detalhar os elementos que constituem o modelo BDI:

Crenças (Beliefs): As crenças representam o conhecimento do agente sobre o ambiente e seu estado atual. Elas são constantemente atualizadas com base nas percepções do agente, refletindo aquilo que o agente acredita ser verdadeiro sobre o mundo. Esse conhecimento é essencial para que o agente possa avaliar corretamente suas opções de ação.

Desejos (Desires): Os desejos são os objetivos ou metas que o agente deseja alcançar. Eles orientam o comportamento do agente, determinando o que ele gostaria de realizar em diferentes situações. Os desejos servem como motivadores para as ações do agente, guiando-o na direção de seus objetivos.

Intenções (Intentions): As intenções são os planos ou cursos de ação que o agente decide seguir para atingir seus desejos. Elas são selecionadas com base nas crenças e desejos do agente e são executadas para alcançar os objetivos definidos. As intenções representam o compromisso do agente com determinadas ações, estabelecendo uma direção clara para o seu comportamento.

Planejamento e Execução: O modelo BDI separa o planejamento da execução das ações. Os agentes BDI podem deliberar sobre diferentes planos e selecionar o mais apropriado com base nas suas crenças e intenções. Após a seleção, os planos são executados de maneira organizada para atingir os objetivos desejados. Essa separação permite que os agentes BDI equilibrem o tempo gasto na deliberação sobre planos e na execução das ações.

Deliberação: O processo de deliberação é utilizado pelo agente BDI para decidir qual plano seguir. Esse processo envolve avaliar os planos disponíveis, considerando as crenças e intenções do agente, e selecionar aquele que melhor atende às suas metas. A deliberação é um componente crítico que assegura que o agente tome decisões informadas e eficazes.

Adaptabilidade: Uma característica importante dos agentes BDI é sua adaptabilidade. Eles são capazes de ajustar suas crenças, desejos e intenções com base nas mudanças no ambiente e nas novas informações que recebem. Essa adaptabilidade é vital para a operação eficaz em ambientes dinâmicos e incertos.

Os agentes cognitivos são baseados em organizações sociais humanas como grupos, hierarquias e mercados. Segundo [Ferber and Gasser, 1991], as principais características dos SMA Cognitivos são as seguintes:

- Representação explícita do ambiente e dos outros agentes da sociedade;
- Podem manter um histórico das interações e ações passadas e, graças a esta memória, são capazes de planejar suas ações futuras;

- Seu sistema de percepção, que permite examinar o ambiente, e o de comunicação, que permite a troca de mensagens entre agentes, são distintos. A comunicação entre agentes é realizada de modo direto, através do envio e recebimento de mensagens;
- Seu mecanismo de controle é deliberativo. Os agentes cognitivos raciocinam e decidem em conjunto sobre quais ações devem ser executadas, que planos seguir e que objetivos devem alcançar;
- Considerando a última asserção, os modelos de organização dos SMA cognitivos são modelos sociológicos, como as organizações humanas;
- Um SMA cognitivo contém, usualmente, poucos agentes, na ordem de algumas dezenas, no máximo.

Essas características possibilitam que os SMA se destaquem em cenários de interação social complexa, onde múltiplos agentes precisam cooperar ou competir para alcançar objetivos comuns. No entanto, além de interações mais abstratas, é importante considerar a aplicação desses conceitos em ambientes reais e físicos.

Os SMA embarcados estendem esses conceitos para sistemas físicos, onde os agentes são implementados diretamente em sistemas embarcados — dispositivos especializados que realizam funções específicas em um ambiente real. Sistemas embarcados são caracterizados por recursos limitados de hardware e software, mas são altamente eficientes para suas tarefas. Quando integrados a SMA, esses sistemas oferecem uma solução distribuída onde múltiplos agentes podem operar de forma autônoma, cada um com seu conjunto de sensores, atuadores e capacidades de processamento, sem depender de um controle centralizado [Ferber, 1999].

Essa abordagem de agentes embarcados traz diversas vantagens, como a descentralização do controle e a capacidade de resposta autônoma dos agentes ao ambiente. Em vez de depender de uma unidade central, cada agente pode tomar decisões localmente e de forma coordenada com outros agentes, contribuindo para a resolução de problemas em tempo real. Essa característica é crucial para aplicações em ambientes distribuídos, como veículos autônomos, redes de sensores ou automação industrial, onde a adaptabilidade e a cooperação são essenciais [Ferber, 1999].

A natureza distribuída desses sistemas proporciona uma gestão mais robusta e escalável de tarefas complexas, permitindo que sistemas dinâmicos e em grande escala, como cidades inteligentes ou sistemas de transporte autônomos, operem de forma eficiente e resiliente. A combinação da arquitetura de SMA com agentes embarcados e o modelo BDI oferece um alto

grau de flexibilidade e adaptabilidade, tornando esses sistemas uma solução promissora para o gerenciamento de soluções em ambientes físicos [Ferber, 1999].

Pode se dizer que um Sistema Multiagentes(SMA) é um framework onde múltiplos agentes autônomos interagem e colaboram, é composto por diversos agentes cognitivos que podem atuar de forma autônoma e proativa em ambiente físico para atingir um objetivo comum [Wooldridge, 2000]. Cada agente, equipado com capacidades de tomada de decisão, contribuem para a resolução de tarefas complexas que estão além do escopo de um único agente, este tipo de sistema é um candidato a reduzir a dependência humana em diversas tarefas.

Um Sistema Multiagente Embarcado consiste em uma coleção de agentes autônomos integrados em dispositivos de hardware, tipicamente com recursos limitados, como sensores, atuadores e capacidade computacional restrita [Brandão et al., 2021]. Esses agentes colaboram ou competem entre si para realizar tarefas específicas no ambiente em que estão inseridos, com alta demanda por eficiência e resposta em tempo real. O uso de sistemas autônomos, como veículos terrestres e embarcações, tem se mostrado promissor para resolver desafios complexos, como a otimização de tarefas logísticas e a limpeza ambiental de ecossistemas aquáticos.

Trabalhando em ambiente físico os agentes necessitam de uma interface de mapeamento e um meio para identificar e perceber tal ambiente para que desta forma o sistema seja efetivo. A percepção do ambiente físico é um desafio complexo, pois envolve a captura de dados brutos de sensores (como câmeras, ultrassom, LiDAR, etc.) e sua transformação em informações significativas e utilizáveis pelos agentes. A interface de mapeamento é importante nesse processo, pois ela traduz esses dados percebidos em uma representação interna que os agentes podem compreender e sobre a qual podem raciocinar. Isso permite que eles identifiquem objetos, obstáculos, outros agentes e até mesmo as condições dinâmicas do ambiente, garantindo que suas ações sejam relevantes e seguras no mundo real [Russell and Norvig, 2010].

Em resumo, enquanto o SMA foca na interação e coordenação entre agentes autônomos para alcançar um comportamento global inteligente, o modelo BDI oferece uma estrutura sólida para implementar o comportamento desses agentes com base em crenças, desejos e intenções. Quando aplicado a sistemas embarcados, essa abordagem torna-se ainda mais poderosa, permitindo que agentes autônomos e distribuídos interajam de forma inteligente e eficiente em ambientes físicos, garantindo uma solução robusta e escalável para problemas complexos [Weiss, 1999]. Dessa forma, essas tecnologias são indispensáveis para a implementação deste projeto.

2.1.1 Framework JaCaMo

O JaCaMo é uma plataforma integrada para o desenvolvimento de SMA, combinando os frameworks Jason, CArtAgO e Moise. Essa integração permite aos desenvolvedores criar agentes inteligentes com comportamento deliberativo (baseado no modelo BDI), que interagem com ambientes artificiais e seguem estruturas organizacionais predefinidas. Por essas características, o JaCaMo é uma escolha estratégica para o desenvolvimento de sistemas complexos de agentes.

O framework oferece suporte completo para a implementação de agentes inteligentes, capazes de tomar decisões baseadas em crenças, desejos e intenções.

A camada Moise permite modelar a organização dos agentes, definindo regras, papéis e relações hierárquicas entre eles. Essa organização é crucial para garantir que os diferentes agentes atuem de forma coordenada e eficiente em tarefas complexas.

A integração com CArtAgO facilita a criação e manipulação de artefatos, que representam os recursos compartilhados pelos agentes no sistema. Esses artefatos podem ser interfaces com sensores, atuadores ou outros recursos do ambiente físico ou virtual. Através da criação de artefatos, os agentes podem interagir de forma contínua com o ambiente, capturando e processando dados em tempo real e manipulando recursos.

A capacidade do JaCaMo de combinar múltiplos paradigmas dentro de um único framework o torna ideal para lidar com a complexidade de projetos onde agentes inteligentes, ambientes interativos e uma organização bem definida são essenciais para o sucesso de um sistema cognitivo.

2.2 Visão Computacional

A visão computacional é a área da ciência que estuda métodos de aquisição, processamento, análise e entendimento de imagens e, no geral, informações do mundo real, com o intuito de produzir dados numéricos ou simbólicos para realizar alguma função ou tomar decisões [Klette, 2014]. A visão computacional encontra aplicações em diversos campos, como reconhecimento de objetos, navegação de robôs, diagnóstico médico, sistemas de vigilância e veículos autônomos. Seus principais componentes atuam de seguinte maneira:

- **Aquisição de Imagem:** O processo de captura de imagens utilizando dispositivos como câmeras digitais ou sensores específicos.

- **Pré-processamento de Imagem:** Envolve a aplicação de técnicas para melhorar a qualidade das imagens capturadas, como remoção de ruído, correção de iluminação e realce de contrastes.
- **Segmentação de Imagem:** A tarefa de dividir a imagem em regiões significativas ou objetos de interesse.
- **Extração de Características:** Identificação e descrição de características importantes nas imagens, como bordas, formas, texturas e cores.
- **Reconhecimento e Interpretação:** Utilização de algoritmos para reconhecer objetos ou padrões nas imagens e interpretar seu significado.

No contexto de sistemas autônomos, a visão computacional é essencial para permitir a percepção do ambiente, incluindo o reconhecimento e diferenciação de objetos, bem como a localização espacial de elementos de interesse. As informações visuais processadas são cruciais para o raciocínio do sistema e a tomada de decisões autônomas.

2.3 Algoritmos de Machine Learning e Deep Learning

O Aprendizado de Máquina (ML) é uma subárea da IA que envolve a construção de algoritmos que podem aprender a partir de dados e fazer previsões ou tomar decisões com base em novos dados. No contexto deste trabalho, ML pode ser utilizado para treinar modelos que identificam e classificam objetos em imagens capturadas pela câmera.

O Aprendizado Profundo (DL) é um subcampo de Machine Learning que utiliza redes neurais artificiais com muitas camadas Deep Neural Networks(DNN). Essas redes são especialmente eficazes para tarefas como reconhecimento de imagem, onde a quantidade de dados e a complexidade das características visuais são elevadas. Em sistemas de visão computacional, DL pode ser utilizado para melhorar a precisão e a eficiência da identificação de objetos, utilizando técnicas como Redes Neurais Convolucionais (CNNs)."

2.4 Tecnologias Utilizadas

Neste projeto, diversas tecnologias foram empregadas para a implementação do sistema de navegação e interação com objetos, combinando técnicas de visão computacional, aprendizado

de máquina e SMA. A seguir, são descritas as principais ferramentas e bibliotecas utilizadas, juntamente com o papel de cada uma no desenvolvimento do sistema.

Python: É uma linguagem de programação que foi escolhida por sua ampla adoção na área de ciência de dados, aprendizado de máquina e visão computacional. Com uma sintaxe clara e uma vasta coleção de bibliotecas como as utilizadas: NumPy e Pandas para manipulação e análise de dados e Matplotlib para visualização dos resultados e métricas de desempenho dos modelos de aprendizado, além disso o Python facilita o desenvolvimento de protótipos e a implementação de algoritmos complexos.

No contexto deste projeto, Python foi utilizado para o desenvolvimento de scripts de processamento de imagens e análise de dados, implementação de algoritmos de aprendizado de máquina e redes neurais e integração de diferentes bibliotecas e frameworks, como OpenCV, TensorFlow e JaCaMo, para criar um sistema coeso.

OpenCV (Open Source Computer Vision Library): é uma biblioteca de programação open source que oferece centenas de funções para processamento de imagens e visão computacional. Será responsável por capturar e manipular imagens e vídeos, aplicar filtros de imagem para remoção de ruído e ajuste de contraste, detectar bordas e contornos de objetos, executar transformações geométricas, como redimensionamento e rotação de imagens e implementar algoritmos de segmentação para identificar objetos de interesse.

TensorFlow: é uma biblioteca open source desenvolvida pelo Google para aprendizado de máquina e redes neurais. É amplamente utilizada em tarefas de visão computacional avançada, como treinamento de modelos de redes neurais convolucionais (CNNs) para reconhecimento de objetos e classificação de imagens, utilização de técnicas de aprendizado profundo para segmentação semântica e detecção de objetos, implementação de modelos pré-treinados, como o MobileNet ou o YOLO (You Only Look Once), para detecção em tempo real e utiliza técnicas de transferência de aprendizado para otimizar o desempenho do modelo sem a necessidade de grandes quantidades de dados rotulados.

IoT: possibilita a interconexão de diferentes dispositivos e sistemas, aumentando a capacidade do agente autônomo de coletar, processar e transmitir dados em tempo real. Isso resulta em maior eficiência operacional, melhor tomada de decisão e maior automação do sistema, reduzindo a necessidade de intervenção humana e aumentando a precisão na tomada de decisão autônoma e interação com o ambiente.

JaCaMo: Conforme detalhado na Seção 2.1.1, o JaCaMo é a plataforma escolhida para

o desenvolvimento do SMA. Ele permite a implementação de agentes inteligentes com lógica BDI, a criação de ambientes interativos via artefatos e a definição de organizações complexas entre agentes. No contexto deste projeto, o JaCaMo é fundamental para coordenar o comportamento dos agentes na navegação e interação do veículo autônomo com o objeto.

Capítulo 3

Identificação e Coleta de Objetos no Ambiente Exógeno

Neste capítulo, será explorado o processo de identificação e interação eficiente de objetos em um ambiente exógeno, com foco na interação entre o sistema desenvolvido e o mundo real. A ideia central é apresentar a metodologia utilizada para mapear o ambiente, reconhecer objetos e implementar ações voltadas à interação e deslocamento do protótipo em direção a um objeto específico.

Serão discutidos os conceitos relacionados à percepção do sistema sobre o ambiente externo e como essas informações são processadas para identificar objetos de interesse. Além disso, será abordada a organização do modelo conceitual, que serve como base para estruturar a interação entre os componentes do sistema, garantindo que as ações tomadas sejam coerentes com os objetivos estabelecidos.

Por fim, o capítulo incluirá uma descrição da abordagem empregada para a interação com o objeto, destacando os desafios e soluções para atuar em um ambiente dinâmico e não controlado. Esses aspectos fornecem uma visão integrada do funcionamento do sistema, estabelecendo as bases para entender como ele opera em cenários reais.

3.1 Metodologia

A metodologia deste projeto é estruturada para garantir a identificação e coleta eficiente de objetos em um ambiente exógeno, integrando técnicas de sistemas multiagentes, visão computacional e API. O objetivo central é mapear o ambiente, identificar o protótipo do veículo e o objeto de interesse, e implementar ações de navegação e interação de forma autônoma e coordenada. A metodologia pode ser melhor vista na figura 1.

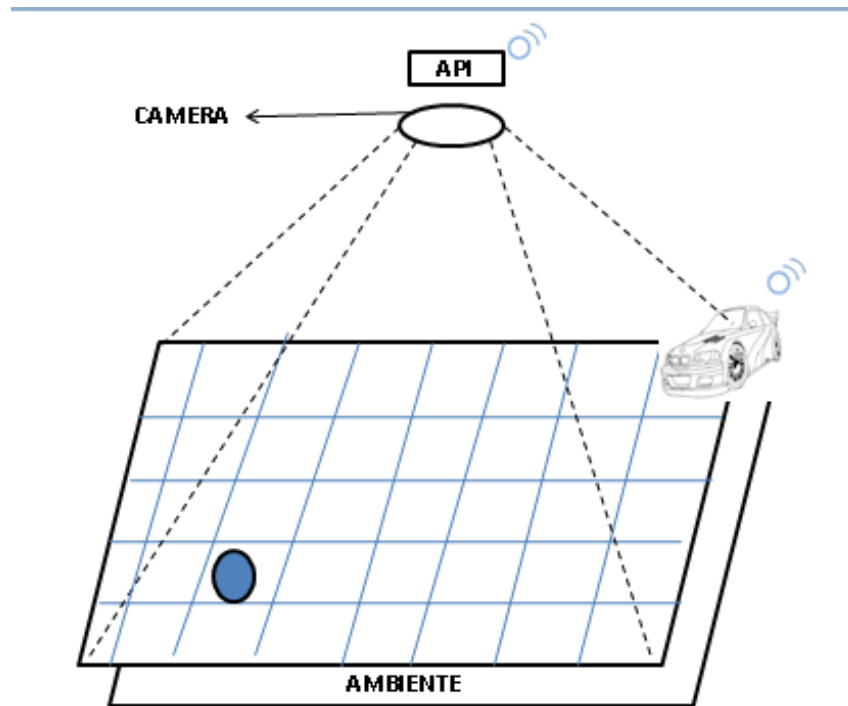


Figura 1: Modelo Conceitual

O processo metodológico compreende diferentes etapas. Inicialmente, realiza-se o planejamento das tecnologias e estratégias, considerando as necessidades de percepção e interação com o ambiente externo. Em seguida, desenvolve-se o sistema multiagente, utilizando o framework JaCaMo (conforme detalhado na Seção 2.1.1), que permite a coordenação entre agentes responsáveis pelo mapeamento, identificação e tomada de decisão.

A visão computacional é empregada para processar dados visuais do ambiente, permitindo que o sistema detecte objetos com base em suas características visuais, como forma e cor. Com base nas informações processadas, o sistema cria um mapeamento dinâmico do ambiente, guiando as ações dos agentes de coleta de forma otimizada.

Essas etapas serão detalhadas nas seções seguintes, destacando como cada componente foi implementado e sua contribuição para o funcionamento integrado do sistema em cenários reais.

1. **Planejamento do Sistema:** Envolve a análise detalhada para definir os requisitos do sistema e as tecnologias mais adequadas às necessidades do projeto, levando em consideração os desafios de operar em um ambiente dinâmico e não controlado. Essa fase contempla a escolha de uma arquitetura distribuída baseada em SMA e a integração de módulos de percepção por visão computacional, com foco na escalabilidade e comunicação entre os componentes.

2. **Desenvolvimento do SMA:** A implementação do SMA é realizada utilizando o framework JaCaMo, que combina agentes autônomos, ambientes artificiais e estruturas organizacionais. Cada agente é projetado com funções específicas, como mapear o ambiente, identificar objetos e coordenar ações de navegação e interação com o objeto. O framework é escolhido por sua flexibilidade e robustez, permitindo uma integração fluida entre os agentes e garantindo que as operações sejam realizadas de maneira organizada e eficiente.
3. **Processamento de Dados com Visão Computacional:** As imagens do ambiente exógeno, capturadas em tempo real, são processadas utilizando algoritmos de visão computacional. Esses algoritmos são configurados para identificar e localizar objetos de interesse com base em características como cor, forma e posição. As informações processadas são repassadas aos agentes do sistema, que utilizam esses dados para tomar decisões e planejar ações, garantindo precisão e eficiência no reconhecimento dos objetos. Para a implementação, será necessário treinar um modelo de visão computacional utilizando a linguagem Python e bibliotecas como o OpenCV e TensorFlow. Esse modelo será capaz de identificar o protótipo de um veículo terrestre e uma bola nas imagens capturadas, fornecendo assim as informações necessárias para os agentes do SMA embarcado.
4. **Mapeamento e Monitoramento:** Com os dados processados, o sistema gera um mapa dinâmico do ambiente, representando a localização e as características dos objetos identificados. Esse mapeamento permite que os agentes ajustem suas ações de forma estratégica, otimizando a cobertura da área e garantindo maior eficiência nas operações de navegação e interação. Essa etapa integra as informações da visão computacional, traduzindo as posições capturadas em um mapa útil para o raciocínio dos agentes e a tomada de decisões, fundamentando as ações subsequentes do sistema.

3.1.1 Arquitetura do Sistema

A arquitetura do sistema proposto é baseada em um modelo distribuído, que utiliza agentes inteligentes para realizar tarefas de identificação, mapeamento e interação com um objeto específico em um ambiente exógeno. O sistema é estruturado em três camadas principais: Camada de Percepção, Camada de Decisão e Camada de Ação, que operam de forma integrada por meio de dispositivos IoT e são coordenadas pelo framework JaCaMo.

Camada de Percepção: É responsável pela coleta de dados do ambiente em tempo real, utilizando sensores (dispositivos IoT) e uma interface de comunicação de dados (API/Middleware). Entre os componentes desta camada, destaca-se:

- **Sensor de profundidade (Camera):** Captura informações sobre localização e características dos objetos (especialmente o protótipo do veículo e a bola) no ambiente exógeno. Os dados coletados são processados (pelo módulo de visão computacional em Python) e disponibilizados aos agentes do sistema por meio da API de comunicação, garantindo a integração entre o hardware de percepção e as demais camadas de processamento e decisão.

Camada de Decisão: o framework JaCaMo é utilizado para gerenciar a comunicação entre os agentes e processar os dados provenientes da camada de percepção. Os principais agentes desta camada incluem:

- **Agente de Processamento de Dados Visuais:** Recebe e organiza os dados dos sensores (coordenadas do veículo e do objeto), descartando informações irrelevantes ou duplicadas para o raciocínio dos agentes.
- **Agente de Planejamento de Ações:** Com base nas informações fornecidas, define estratégias para otimizar a navegação e interação do protótipo com o ambiente, maximizando a eficiência na aproximação e no engajamento com o objeto, e ajustando ações conforme necessário.

Camada de Ação: É onde as decisões tomadas pela Camada de Decisão são executadas, impactando diretamente o ambiente físico. Os agentes desta camada incluem:

- **Agente de Controle do Veículo:** Coordena os mecanismos de movimentação e interação com os objetos no ambiente (como os atuadores do veículo), ajustando as ações para garantir a eficiência operacional e o sucesso na interação com o alvo.
- **Agente de Comunicação IoT:** Garante a troca de dados entre os dispositivos IoT (como a câmera e os motores do veículo) e os agentes do sistema, transmitindo informações de forma contínua e em tempo real, e enviando comandos de controle para o hardware.

Comunicação entre Agentes: A interação entre os agentes é realizada por meio de mensagens trocadas dentro do framework JaCaMo. Cada agente desempenha um papel específico

de forma autônoma, mas a coordenação é viabilizada por protocolos usando o KQML, que é nativo do Jason.

Essa comunicação eficiente entre as camadas e os agentes assegura que o sistema seja capaz de reagir rapidamente a mudanças no ambiente exógeno, adaptando-se às condições dinâmicas para atingir seus objetivos de identificação e interação com o objeto.

3.2 Identificação e Interação com o Objeto

A sessão anterior (3.1 Metodologia) estabeleceu as bases de como a arquitetura do sistema é organizada em camadas de Percepção, Decisão e Ação. Agora, aprofundaremos nas capacidades específicas de identificação e interação com o objeto de interesse, que são cruciais para o funcionamento autônomo do protótipo do veículo. Esta seção detalhará o processo de percepção visual do ambiente, a aplicação de algoritmos avançados para detecção de objetos e o subsequente envio dessas informações para os agentes deliberativos. O foco principal será como o sistema utiliza dados visuais para localizar o protótipo do veículo terrestre e a bola, e como essa percepção informará as ações de navegação e engajamento.

3.2.1 Monitoramento com Câmera

O monitoramento com câmera é responsável por capturar imagens em tempo real do ambiente exógeno para identificar e localizar o protótipo do veículo e a bola. Essa funcionalidade permite que o sistema reconheça os elementos presentes no cenário e tome decisões de navegação e interação baseadas em informações visuais precisas.

A câmera utilizada no projeto estará posicionada estrategicamente para maximizar o campo de visão e garantir a cobertura completa do ambiente monitorado. No contexto deste projeto, a câmera estará localizada a dois metros do solo, permitindo uma visão superior que facilita a detecção e o rastreamento tanto do veículo autônomo quanto do objeto (bola) dentro da área de operação.

As imagens capturadas são transferidas para o sistema em tempo real, utilizando uma interface de comunicação eficiente (API/Middleware) que conecta a câmera aos demais componentes da Camada de Percepção e Decisão. Esse fluxo contínuo de dados visuais é crucial para que o sistema possa reagir rapidamente às mudanças no ambiente, fornecendo informações atualizadas sobre as posições relativas do veículo e da bola.

Os dados coletados pela câmera são processados por algoritmos de visão computacional, que analisam as imagens e identificam o veículo autônomo e o objeto de interesse (a bola) com base em suas características visuais, como forma, tamanho, cor e posição em coordenadas. Esses algoritmos utilizam técnicas avançadas de DL para aumentar a precisão e eficiência do reconhecimento.

Entre as técnicas avançadas de aprendizado de máquina que podem ser aplicadas, destacam-se:

- **Redes Neurais Convolucionais (CNNs):** São arquiteturas de DL altamente eficazes para tarefas de classificação e detecção de objetos em imagens. Modelos como ResNet, Inception ou VGG podem ser treinados ou utilizados via transferência de aprendizado para reconhecer padrões visuais específicos do protótipo do veículo e da bola, mesmo em diferentes condições.
- **Algoritmos de Detecção de Objetos em Tempo Real (Single-Shot Detectors):** Para garantir uma detecção rápida, essencial em sistemas autônomos, algoritmos como YOLO (You Only Look Once) ou SSD (Single Shot MultiBox Detector) são excelentes opções. Eles são capazes de identificar e localizar múltiplos objetos em uma única passagem pela rede neural, fornecendo caixas delimitadoras e classes para o carro e a bola em tempo quase real.
- **Segmentação Semântica ou por Instância:** Embora mais complexas, técnicas como Mask R-CNN poderiam ser exploradas para uma compreensão ainda mais detalhada do ambiente, não apenas localizando os objetos, mas também segmentando precisamente suas formas em nível de pixel, o que pode ser útil para navegação em espaços restritos ou para interações mais complexas.

Por fim, desafios inerentes a ambientes exógenos, como condições de iluminação variáveis (sombras, brilho excessivo) ou obstruções no campo de visão (objetos temporariamente no caminho), são abordados por meio de ajustes automáticos da câmera (ex: controle de exposição e balanço de branco) e de técnicas de pré-processamento de imagem (ex: equalização de histograma, remoção de ruído). Essas estratégias garantem que o sistema mantenha sua eficiência e robustez na percepção, mesmo em cenários dinâmicos e imprevisíveis.

3.2.2 Desafios e Robustez da Percepção Visual

A atuação de um sistema autônomo em um ambiente físico, ou exógeno, impõe desafios significativos à sua camada de percepção. A imprevisibilidade e o dinamismo inerentes a esses cenários exigem que os algoritmos de visão computacional sejam robustos o suficiente para lidar com variáveis que não podem ser completamente controladas ou previstas. A robustez da percepção visual é, portanto, um fator crítico para a confiabilidade e o sucesso do sistema.

Neste trabalho, a robustez da camada de percepção foi abordada por meio de um modelo de aprendizado de máquina capaz de lidar com os seguintes desafios:

- **Variações de Iluminação:** A intensidade e a direção da luz em um ambiente real variam constantemente devido a fatores como luz natural, sombras e reflexos. Um algoritmo de visão não robusto poderia falhar na detecção da bola ou do veículo sob diferentes condições de luminosidade. O modelo de aprendizado profundo, treinado com um vasto conjunto de dados que inclui imagens capturadas sob diversas condições de luz, aprendeu a identificar as características dos objetos independentemente dessas variações, garantindo sua detecção em diferentes cenários.
- **Oclusões Parciais:** Em um ambiente dinâmico, os objetos podem ser temporariamente ocluídos por outros elementos. A bola, por exemplo, pode ser parcialmente coberta pelo veículo ou por um obstáculo. A arquitetura de rede neural convolucional (CNN) utilizada no modelo é inerentemente robusta a esse desafio, pois aprende a identificar padrões e características dos objetos mesmo quando apenas uma parte deles é visível.
- **Ruído e Fundo Dinâmico:** Ambientes exógenos contêm uma grande quantidade de elementos visuais que não são de interesse para o sistema (ruído). A capacidade do modelo em diferenciar a bola e o veículo de outros objetos e texturas do ambiente é crucial para evitar falsos positivos e garantir que a atenção do sistema seja direcionada apenas aos objetos relevantes para a tarefa.

A integração dessas técnicas de aprendizado profundo na camada de percepção garante que o sistema não apenas "veja" o ambiente, mas o compreenda de maneira confiável, fornecendo dados de localização consistentes e precisos para a camada de decisão. A validação dessa robustez é um dos pilares da prova de conceito deste projeto.

3.2.3 Algoritmos de Detecção

A identificação e localização do protótipo do veículo autônomo e da bola no ambiente exógeno são realizadas por meio de algoritmos avançados de ML e DL, que garantem precisão e eficiência no reconhecimento desses elementos cruciais para a navegação e interação do sistema.

Inicialmente, algoritmos de ML podem ser empregados para realizar análises preliminares nos dados visuais capturados pela câmera. Embora os modelos de DL sejam a base da detecção robusta, técnicas de ML mais simples podem auxiliar em tarefas como a segmentação inicial de regiões de interesse ou a filtragem de ruídos e informações irrelevantes, otimizando a qualidade dos dados antes de serem submetidos aos modelos mais complexos. Essa pré-análise ajuda a reduzir a complexidade computacional e a focar o processamento nas áreas mais promissoras.

O DL, por sua vez, constitui o cerne da capacidade de reconhecimento do sistema, utilizando DNN, especialmente as CNNs. Essas redes são notavelmente eficazes para o reconhecimento e a localização de objetos em imagens e vídeos. Elas são treinadas com um extenso conjunto de dados contendo múltiplos exemplos do protótipo do veículo e da bola em diversas condições de iluminação, ângulos e distâncias. Através desse treinamento, o modelo aprende a distinguir e classificar esses objetos com alta precisão, identificando características complexas como bordas, texturas, padrões de cor e formas que os diferenciam de outros elementos no ambiente.

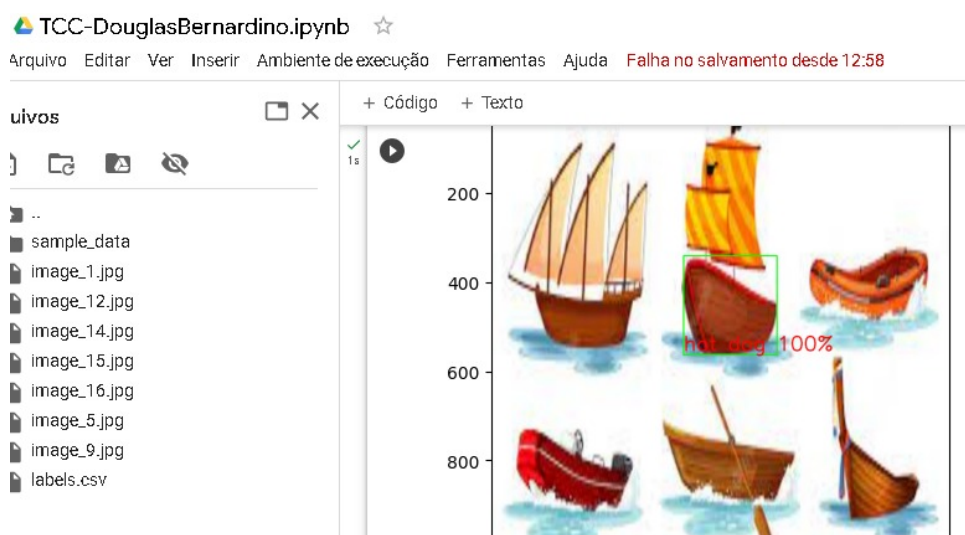


Figura 2: Modelo Pré Treinado

A escolha de um algoritmo de detecção robusto, como o YOLO, foi apenas o primeiro passo. A qualidade da percepção visual é intrinsecamente ligada à especificidade dos dados de treina-

mento. Modelos pré-treinados em grandes bases de dados genéricas frequentemente falham ao serem aplicados a cenários específicos e não convencionais, como o ambiente exógeno do protótipo. A Figura 2 ilustra este desafio. No teste, o modelo pré-treinado demonstrou um **erro de generalização** notório, classificando o casco de uma embarcação como um objeto não relacionado ('hotdog') outro erro notório foi identificar um carro como um hidrante. Diante disso, o projeto se concentrou na criação de um dataset próprio e dedicado para garantir a precisão e a baixa Taxa de Falsos Positivos (TFP).

Fluxo de Processamento da Detecção:

1. **Aquisição e Pré-processamento:** As imagens são capturadas pela câmera em tempo real e passam por etapas iniciais de pré-processamento, conforme detalhado na Seção 3.2.1, para otimização da qualidade visual (ex: ajuste de iluminação, remoção de ruído)..
2. **Pré-análise (ML):** Modelos de ML podem realizar uma pré-análise rápida para refinar os dados, descartando áreas da imagem que são claramente irrelevantes ou direcionando o foco para potenciais objetos de interesse, se necessário para otimização de desempenho.
3. **Detecção e Classificação (DL):** O modelo de Deep Learning (ex: YOLO, SSD) processa as imagens (ou regiões de interesse) e realiza a detecção e classificação dos objetos em tempo real. Para cada objeto identificado (o protótipo do veículo e a bola), o modelo atribui uma categoria (ex: "carro", "bola"), um nível de confiança e as coordenadas de sua localização espacial na imagem (geralmente uma caixa delimitadora).

Esses algoritmos de detecção garantem que o sistema possa identificar o veículo e a bola de maneira autônoma e precisa, mesmo em cenários dinâmicos e não controlados. Essa identificação eficiente é fundamental para o envio de informações espaciais exatas aos agentes na Camada de Decisão, permitindo que as ações subsequentes de navegação e interação sejam bem fundamentadas e eficazes.

Os algoritmos de detecção de objetos são fundamentais para a camada de percepção, pois permitem que o sistema não apenas reconheça, mas também localize os objetos de interesse no campo de visão. **Para o objetivo de *Mapeamento do Ambiente Exógeno* deste trabalho, a capacidade de localização é o fator crítico.**

Dentre as diversas abordagens na Visão Computacional, foi adotada a arquitetura YOLO, que se destaca por sua alta velocidade de inferência, característica essencial para o processamento em tempo real exigido por sistemas robóticos embarcados.

A funcionalidade crucial do algoritmo de detecção para este projeto é a geração da bounding box(caixa delimitadora). Diferentemente de modelos que apenas classificam uma imagem como um todo, o YOLO retorna um conjunto de dados que, além da classe do objeto ('carro', 'bola') e o nível de confiança, fornece as coordenadas espaciais (X, Y, largura e altura) precisas do objeto na imagem.

É a partir do centro dessas bounding boxes que o sistema extrai o dado bruto de localização, que será processado pela Discretização de Coordenadas (conforme detalhado na Seção 4.2.2). Desta forma, o algoritmo cumpre o papel de traduzir a informação visual contínua em dados posicionais concretos e estruturados para o sistema multiagente.

3.2.4 Envio de Dados para o Agente

O envio das informações visuais para os agentes é uma etapa crítica, garantindo que a Camada de Decisão receba dados atualizados e precisos da Camada de Percepção. Este processo detalha como as coordenadas e características do protótipo do veículo autônomo e da bola, identificadas pelos algoritmos de detecção, são empacotadas e disponibilizadas para os agentes do sistema, especificamente através dos artefatos do CArtAgO no framework JaCaMo.

Processo de Disponibilização de Dados para os Agentes:

1. **Coleta e Formatação das Informações Visuais:** Após o processamento pelas técnicas de visão computacional e DL(conforme 3.2.2), as informações sobre a localização (coordenadas X, Y), tamanho e identidade (ex: "carro", "bola") dos objetos de interesse são agrupadas e formatadas. Este módulo de visão implementado em Python, prepara os dados para serem consumidos pelos agentes.
2. **Transmissão para o Ambiente JaCaMo via Artefatos CArtAgO:** A ponte entre o módulo de visão computacional (Python) e o sistema de agentes (JaCaMo) é realizada por meio de artefatos CArtAgO. Estes artefatos funcionam como recursos compartilhados e interfaces com o ambiente. O módulo de visão externa, por exemplo, pode "escrever" as coordenadas atualizadas do carro e da bola em propriedades ou operações de um artefato CArtAgO dedicado à percepção. Essa comunicação pode ser implementada via uma API de comunicação (conforme 3.1.1) que o JaCaMo possa interagir (e.g., uma interface REST, sockets, ou uma biblioteca de comunicação customizada para CArtAgO).
3. **Incorporação como Crenças dos Agentes:** Uma vez que os dados são atualizados no

artefato CArtAgO, os agentes JaCaMo na Camada de Decisão podem "perceber" essas mudanças. Os agentes são configurados para observar o estado desses artefatos. Ao detectar uma nova informação (ex: uma nova coordenada da bola), essa informação é imediatamente incorporada às suas bases de crenças (beliefs). Por exemplo, um agente pode ter a crença $posicaobola(X, Y)$ que é atualizada constantemente com os dados provenientes do artefato. Isso permite que os agentes tenham um conhecimento atualizado do ambiente em tempo real.

Benefícios da Abordagem com Artefatos CArtAgO:

- **Abstração da Complexidade:** Os artefatos CArtAgO abstraem os detalhes de comunicação e hardware, permitindo que os agentes se concentrem puramente na lógica de decisão (BDI).
- **Acesso em Tempo Real às Crenças:** A atualização direta das crenças dos agentes a partir dos artefatos garante que as deliberações e o planejamento sejam baseados nas informações mais recentes do ambiente, resultando em baixa latência para a tomada de decisões.
- **Modularidade e Manutenibilidade:** Separa a lógica de percepção e comunicação de baixo nível do raciocínio dos agentes, tornando o sistema mais modular e fácil de manter ou estender.

Essa integração direta, utilizando a capacidade do JaCaMo de interagir com o ambiente através de artefatos CArtAgO, possibilita que o sistema atue de forma coordenada e eficiente. Os agentes respondem prontamente às mudanças na posição do veículo e da bola, fundamentando suas crenças e, consequentemente, seus planos e ações em tempo real para o objetivo de navegação e interação.

3.3 Modelo Conceitual

O modelo conceitual do sistema é estruturado com base no paradigma BDI, que define o comportamento dos agentes de acordo com suas crenças, desejos e intenções. Essa abordagem permite que os agentes ajam de forma racional e proativa em ambientes dinâmicos. No contexto deste projeto, cada agente, especialmente o Agente Carro, possui um papel específico no sistema e é guiado por esses conceitos fundamentais para atuar de forma autônoma e coordenada no ambiente exógeno, visando a identificação e interação com a bola.

Neste modelo, a capacidade do sistema de perceber o ambiente (através da visão computacional, como visto em 3.2.1 e 3.2.2) e processar as informações visuais (conforme 3.2.3) se traduz diretamente nas crenças dos agentes. As crenças representam o conhecimento que o agente possui sobre o estado atual do ambiente, como a posição do próprio veículo, a localização da bola, e quaisquer obstáculos relevantes.

Com base nessas crenças, os agentes formulam seus desejos, que são os objetivos a serem alcançados (e.g., "aproximar-se da bola", "interagir com a bola"). A partir dos desejos e das crenças atuais, os agentes deliberam e selecionam as intenções, que são os planos de ação que se comprometem a executar. As intenções guiam a sequência de comandos que serão enviados aos atuadores do veículo, permitindo a navegação e o engajamento físico com o objeto.

A coordenação entre os diferentes agentes, facilitada pelo framework JaCaMo, assegura que o sistema opere de forma coesa. Este modelo BDI é a base para a inteligência e adaptabilidade do nosso protótipo, permitindo que ele tome decisões autônomas e execute tarefas complexas de interação no mundo real.

3.3.1 Definição de Crenças, Intenções, Desejos e Planos

Cada agente possui um papel específico no sistema e é guiado por esses conceitos fundamentais para atuar de forma autônoma e coordenada no ambiente exógeno, visando a identificação e interação com o objeto (bola). A aplicação do modelo BDI permite que o sistema adapte-se dinamicamente às condições do ambiente e alcance seus objetivos de forma proativa.

Agente de Percepção

Este agente é a interface do sistema com o mundo real, traduzindo dados brutos dos sensores em informações úteis para os demais agentes.

- **Crenças:**

- Coordenadas atualizadas do protótipo do veículo autônomo (carro) e da bola no ambiente.
- Características visuais (cor, forma, tamanho) dos objetos detectados.
- Estado atual do campo de visão da câmera (presença de obstruções, condições de iluminação).

- **Desejos/Intenções:**

- Garantir a coleta contínua e precisa de dados visuais do ambiente.
- Identificar e localizar o protótipo do veículo e a bola em tempo real.

- **Planos:**

- Processar imagens capturadas pela câmera/sensor de profundidade utilizando algoritmos de visão computacional (conforme 3.2.1 e 3.2.2).
- Atualizar continuamente os artefatos CArtAgO com as coordenadas e estados do carro e da bola, disponibilizando essas informações para os agentes da Camada de Decisão.
- Aplicar técnicas de pré-processamento de imagem para otimizar a qualidade dos dados visuais.

Agente de Decisão

Responsável por interpretar as percepções, formular objetivos e planejar a sequência de ações mais eficaz para a navegação e interação.

- **Crenças:**

- Posição exata do protótipo do veículo (X_{carro} , Y_{carro}).
- Posição exata da bola (X_{bola} , Y_{bola}).
- Distância e ângulo entre o protótipo do veículo e a bola.
- Status de interação atual (se o veículo já está interagindo ou se aproximando).
- Qualquer obstáculo relevante no caminho identificado pelo Agente de Percepção.

- **Desejos/Intenções:**

- Aproximar o protótipo do veículo da bola.
- Iniciar e manter a interação com a bola (ex: "empurrar", "interceptar").
- Otimizar a rota de navegação para alcançar a bola eficientemente.

- **Planos:**

- Analisar as crenças sobre a posição do carro e da bola para determinar a melhor rota de aproximação.
- Decidir a sequência de movimentos (avançar, girar, parar) necessária para atingir a bola.
- Enviar comandos de navegação específicos para o Agente de Ação (ex: move(frente), gira(direita, angulo)).
- Ajustar planos em tempo real com base em mudanças na posição da bola ou do ambiente.

Agente de Ação

Este agente é o executor, responsável por converter os planos da Camada de Decisão em comandos físicos para os atuadores do veículo.

- **Crenças:**

- Comandos de movimentação recebidos do Agente de Decisão (ex: moverpara(x,y), girar(graus)).
- Feedback dos atuadores do veículo (ex: "movimento concluído", "obstáculo detectado pelo atuador").
- Capacidade operacional dos motores e mecanismos de movimentação.

- **Desejos/Intenções:**

- Executar fielmente os comandos de navegação e interação recebidos.
- Garantir a movimentação suave e precisa do protótipo do veículo.
- Responder a falhas ou condições inesperadas durante a execução dos movimentos.

- **Planos:**

- Traduzir os comandos de alto nível do Agente de Decisão em sinais específicos para os atuadores do veículo (motores, servos).
- Monitorar o progresso da execução dos movimentos e reportar o status de volta ao Agente de Decisão.
- Ajustar a velocidade ou direção do veículo conforme necessário para evitar pequenos obstáculos ou manter o rumo em tempo real.

Integração entre os Agentes

Os agentes trabalham de forma colaborativa e cíclica, trocando informações e ajustando suas crenças, desejos e planos conforme novas percepções do ambiente. O fluxo se inicia na percepção, que alimenta as crenças dos agentes de decisão, levando à formulação de desejos e seleção de intenções/planos. Estes, por sua vez, são traduzidos em ações pelo Agente de Ação, que impactam o ambiente físico. Essa abordagem dinâmica assegura que o sistema seja capaz de lidar com ambientes exógenos e não estruturados, garantindo eficiência nas tarefas de navegação e interação com a bola.

Essa definição clara de crenças, desejos e planos para cada agente forma a base do modelo conceitual e permite uma operação integrada e adaptativa, culminando na capacidade do protótipo de atingir o objetivo de interação com o objeto de forma autônoma.

Essa definição clara de crenças, intenções e planos para cada agente forma a base do modelo conceitual e permite uma operação integrada e adaptativa.

3.3.2 Implementação do Agente Carro

O Agente Carro desempenha um papel central no sistema, sendo responsável pela execução das ações no ambiente exógeno, especificamente a navegação e interação com a bola. Este agente utiliza tecnologias de controle autônomo para operar de maneira eficiente e coordenada com os demais agentes do sistema, conforme os princípios do paradigma BDI.

Estrutura do Agente Carro

O Agente Carro é concebido com uma estrutura que reflete suas responsabilidades na arquitetura de três camadas (Percepção, Decisão e Ação), embora sua implementação direta esteja focada na execução dos planos de ação. Suas principais capacidades ou funções são:

- **Capacidade de Percepção (no contexto do agente):** Esta capacidade refere-se à habilidade do Agente Carro de receber e interpretar dados enviados pelo Agente de Decisão, que por sua vez obteve essas informações da Camada de Percepção (câmera, sensores). As informações incluem a localização do protótipo do veículo e da bola, além de possíveis obstáculos detectados.
- **Capacidade de Decisão (no contexto do agente):** Implementada utilizando o framework JaCaMo, esta capacidade permite ao Agente Carro processar as informações recebidas do

ambiente (via suas crenças) e os planos de alto nível gerados pelo Agente de Planejamento de Ações (da Camada de Decisão). Ele traduz esses planos em sequências de movimentos e ações específicas. Ele também é responsável por utilizar algoritmos de desvio de obstáculos e recalcular trajetos em tempo real caso a percepção indique uma mudança no ambiente imediato.

- **Capacidade de Ação:** Esta capacidade é o elo direto com o hardware. Ela controla os atuadores do veículo, incluindo motores e o sistema de interação (se houver um mecanismo específico para interagir com a bola além do próprio movimento do carro). Esta capacidade executa os planos de baixo nível gerados pela Capacidade de Decisão, garantindo que o carro siga as rotas planejadas e realize a interação com a bola conforme o objetivo.

Funcionamento do Agente Carro

O funcionamento do Agente Carro ocorre em etapas cíclicas e contínuas, alinhadas com o ciclo percepção-decisão-ação:

1. **Recepção de Comandos e Dados:** O agente recebe comandos e informações estratégicas do Agente de Decisão. Isso inclui a localização alvo da bola, a posição atual mapeada do veículo, e os planos de navegação definidos para alcançar o objetivo.
2. **Execução do Planejamento da Rota e Desvio de Obstáculos:** Com base nos comandos recebidos, o Agente Carro utiliza algoritmos internos de controle para executar o caminho mais eficiente até a bola. Durante o trajeto, ele ajusta a rota dinamicamente em caso de detecção de obstáculos próximos ou mudanças inesperadas no ambiente percebidas por seus próprios sensores de proximidade (se existirem, além da câmera principal).
3. **Interação com a Bola:** Ao alcançar a localização da bola e estar na posição de engajamento, o sistema de interação é ativado. Este sistema pode envolver a movimentação direcionada do próprio veículo para empurrar ou interceptar a bola, ou a ativação de um mecanismo específico para essa finalidade. O objetivo é manipular a bola conforme a estratégia definida.
4. **Atualização do Estado:** Após a tentativa de interação, o Agente Carro envia um relatório para o Agente de Decisão, informando o status da operação (ex: "bola alcançada", "in-

teração concluída", "obstáculo encontrado") e dados atualizados de sua própria posição. Caso haja falhas ou problemas, o agente pode solicitar novas instruções ou iniciar um plano de recuperação.

Tecnologias Utilizadas na Implementação do Agente Carro

A implementação do Agente Carro faz uso das seguintes tecnologias para garantir sua funcionalidade autônoma:

- **Framework JaCaMo:** Utilizado para gerenciar a lógica BDI do Agente Carro, sua comunicação e coordenação com os demais agentes do sistema, como o Agente de Percepção e o Agente de Decisão.
- **Sistemas de Controle de Veículo:** Engloba os componentes de software e hardware que traduzem os planos de ação em comandos elétricos para os atuadores do protótipo (motores de tração, sistemas de direção), permitindo a movimentação.
- **Mecanismos de Interação:** Refere-se a qualquer dispositivo ou estratégia mecânica acoplada ao veículo (ou ao próprio movimento do carro) projetada para engajar e manipular a bola.
- **Comunicação IoT (Interna ao Protótipo):** Permite o envio e recepção de dados em tempo real entre o componente de processamento do Agente Carro e seus atuadores/sensores internos, utilizando protocolos eficientes para garantir baixa latência na execução dos movimentos.

Integração com o Sistema

O Agente Carro está totalmente integrado às outras camadas do sistema. Ele atua em sinergia com o Agente de Percepção (que alimenta o Agente de Decisão) e o Agente de Decisão, garantindo que as ações realizadas sejam alinhadas com os objetivos globais de navegação e interação com a bola. Essa integração é feita por meio de mensagens trocadas dentro do framework JaCaMo, utilizando o protocolo Agent Communication Language (ACL) para a comunicação entre agentes de software e, por meio de artefatos CArtaGO, a interação com o ambiente físico.

Desafios e Soluções

A implementação do Agente Carro enfrentou alguns desafios inerentes a sistemas autônomos em ambientes dinâmicos:

- **Navegação em Ambiente Dinâmico:** A necessidade de lidar com mudanças constantes na posição da bola e em potenciais obstáculos foi resolvida pela integração contínua com a visão computacional e o monitoramento em tempo real do ambiente, permitindo que o Agente de Decisão recalcule planos e o Agente Carro execute desvios.
- **Conectividade e Latência:** Problemas de latência na comunicação entre o sistema de controle do Agente Carro e seus atuadores foram minimizados utilizando protocolos de comunicação eficientes e leves, assegurando que os comandos cheguem rapidamente ao hardware.
- **Precisão na Interação:** Para garantir o engajamento eficiente com a bola, o sistema de controle de movimento e os mecanismos de interação são projetados com base em algoritmos de controle de feedback que ajustam a abordagem em tempo real, considerando a forma e o movimento da bola.

Com essa estrutura, o Agente Carro desempenha um papel essencial na execução das ações do sistema, operando de maneira eficiente e adaptativa em ambientes exógenos para alcançar o objetivo de interagir com a bola.

Capítulo 4

Implementação

Esta seção apresenta uma análise crítica do projeto, abordando as limitações encontradas durante o processo de desenvolvimento e os resultados preliminares obtidos. A discussão é focada na validação da arquitetura de percepção e na comunicação com os agentes, que constitui o principal escopo de contribuição deste trabalho. Além disso, são delineados os caminhos para a evolução futura do sistema, expandindo as capacidades do protótipo em direção à sua implementação completa.

4.1 Limitações Atuais

A concepção original do sistema de veículo autônomo, conforme detalhado nos capítulos anteriores, abrange um ciclo completo de percepção, decisão e ação. No entanto, o desenvolvimento e a integração de todas as camadas em um protótipo físico robusto apresentaram desafios significativos, especialmente em relação à complexidade e ao tempo necessário para a implementação completa.

Dessa forma, o escopo deste trabalho foi delimitado para a validação e prova de conceito da Camada de Percepção (visão computacional) e da interface de comunicação de dados com a Camada de Decisão (API). As limitações atuais, que definem o recorte do projeto para esta defesa, incluem:

- **Implementação Parcial da Camada de Ação:** Embora o modelo conceitual e o agente de controle do veículo tenham sido projetados, a implementação física e a calibração precisa da camada de ação (movimentação e interação com a bola) não foram totalmente concluídas. O sistema de controle de movimento e os atuadores foram modelados, mas sua integração e testes em um ambiente físico complexo são propostos como etapas futuras.
- **Foco em Resultados de Simulação e Prova de Conceito:** Os resultados apresentados concentram-se na validação da precisão dos algoritmos de detecção, classificação e loca-

lização de coordenadas, bem como na eficiência da comunicação de dados. A demonstração da navegação autônoma em um ambiente físico dinâmico completo, embora seja o objetivo final, foi tratada como uma perspectiva para futuras pesquisas.

- **Ausência de Detalhamento Técnico das Bibliotecas:** Para manter o foco na metodologia e nos resultados funcionais do sistema, e por uma questão de escopo, o detalhamento exaustivo da implementação de bibliotecas de aprendizado de máquina (como TensorFlow ou OpenCV) não foi o objetivo principal da discussão. O trabalho enfatiza o resultado da aplicação dessas tecnologias, e não o processo de configuração e programação em si.

Essas limitações não diminuem a contribuição do trabalho, mas sim demarcam claramente o foco da pesquisa e o escopo dos resultados que serão apresentados. A validação bem-sucedida das camadas de percepção e comunicação serve como a base fundamental para a continuidade da pesquisa e o desenvolvimento futuro do sistema completo.

4.2 Validação e Detalhamento da Camada de Percepção Visual: A Contribuição Efetiva

Esta seção apresenta a análise detalhada dos resultados obtidos na implementação da Camada de Percepção Visual e da API de Comunicação, que constituem a contribuição prática central deste Trabalho de Conclusão de Curso. As demais etapas da arquitetura (Camadas de Decisão e Ação) são discutidas no Capítulo 5 como Trabalho Futuro.

O foco é demonstrar a prova de conceito de um sistema robusto, capaz de identificar e localizar com precisão objetos de interesse em um ambiente exógeno e de fornecer esses dados de forma estruturada para os agentes BDI.

4.2.1 Detalhamento do Processo de Detecção e Localização de Objetos

Os testes preliminares validaram a eficácia dos algoritmos de classificação e reconhecimento de objetos.

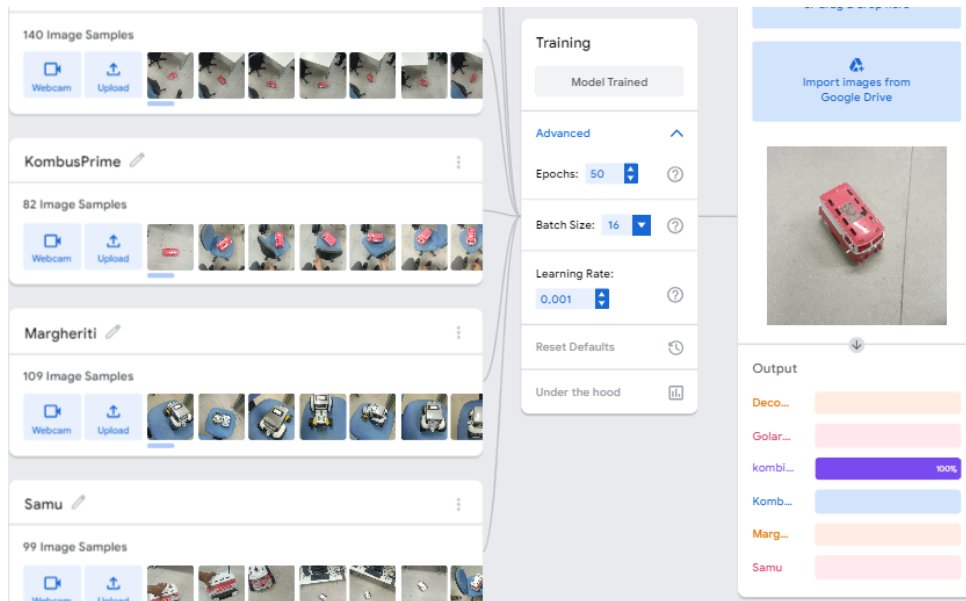


Figura 3: DataSet com Tensorflow para Classificação

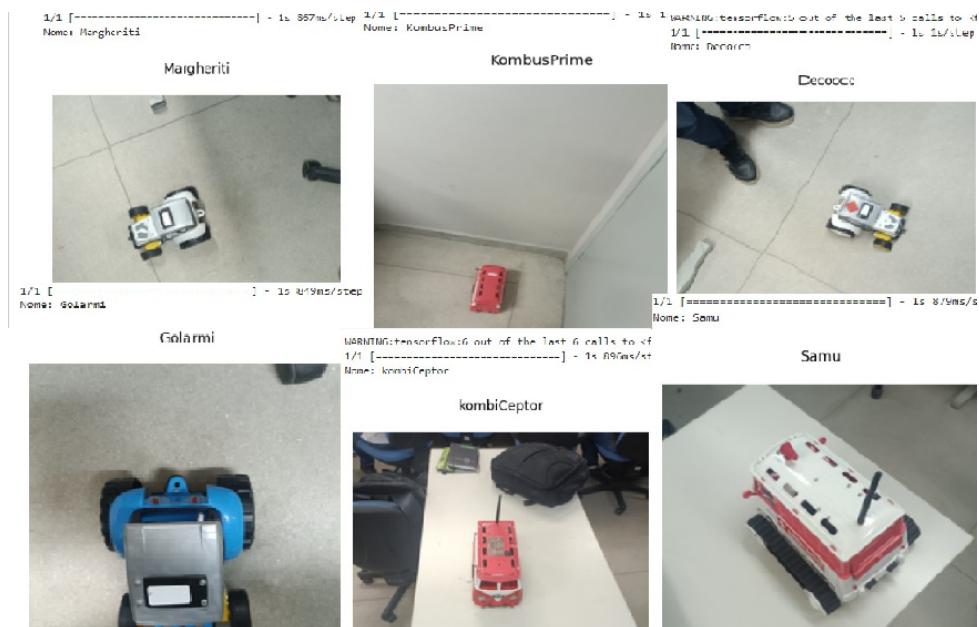


Figura 4: Modelo de Classificação com Tensorflow

A Figura 3 ilustra o processo de criação do DataSet do modelo de classificação na plataforma TensorFlow que gera modelos simplificados de aprendizado profundo capaz de identificar a bola e o carro, a figura 4 apresenta o funcionamento do código em python para o reconhecimento da imagens.

A etapa de percepção visual foi validada utilizando um conjunto de dados de teste contendo imagens do protótipo do veículo autônomo e da bola em diferentes condições. A eficácia do sistema em identificar e classificar corretamente os objetos foi avaliada por meio das seguintes

métricas:

1. **Taxa de Detecção Correta(TDC):** O sistema obteve uma TDC de 96%, indicando que a grande maioria dos objetos presentes nas imagens foi corretamente identificada e classificada.
2. **Taxa de Falsos Positivos(TFP):** A TFP foi de apenas 4%, demonstrando a robustez dos algoritmos em evitar a identificação incorreta de objetos irrelevantes no ambiente.

O sucesso do sistema autônomo depende da capacidade de a Camada de Percepção fornecer as coordenadas espaciais dos objetos. Para tal, o foco foi na Detecção e Localização de Objetos, e não apenas na classificação simples.

Metodologia e Processo de Treinamento:

1. **Arquitetura:** Em vez de modelos de classificação, utilizou-se uma arquitetura baseada em YOLO. Esta arquitetura foi selecionada por sua alta velocidade de inferência e capacidade de retorno de caixas delimitadoras (bounding boxes) em tempo real, características essenciais para a robótica.
2. **Captura de Dados e Anotação:** A plataforma Roboflow foi utilizada para o gerenciamento e anotação manual das imagens. Neste processo, foram coletadas 1.295 imagens sob diferentes condições (ângulos, distância, variações de iluminação) e caixas delimitadoras foram desenhadas em torno do protótipo do veículo e da bola, fornecendo os dados de localização espacial.

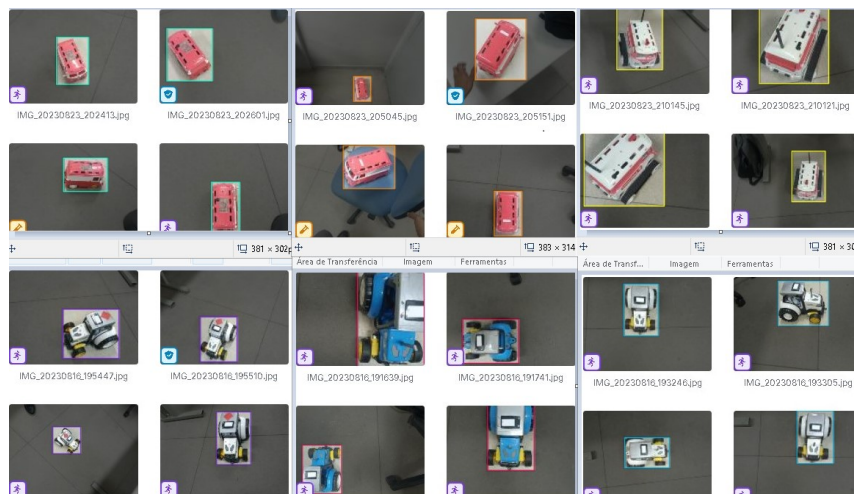


Figura 5: Gerenciamento e Anotação Manual das Imagens

Para garantir a generalização do modelo e evitar o superajuste (overfitting) aos dados de treinamento, o conjunto total de imagens foi dividido em três subconjuntos: Treinamento, Validação e Teste. A Figura 6 ilustra a divisão realizada na plataforma Roboflow. A utilização de um conjunto de Validação e Teste separado foi fundamental para medir a precisão do modelo em dados não vistos, garantindo que as métricas de desempenho apresentadas (como o TDC) são estatisticamente válidas para o ambiente exógeno.



Figura 6: Divisão de Conjunto de Dados

3. **Validação Visual:** A Figura 7 ilustra a prova de conceito do modelo de detecção treinado. A imagem demonstra a capacidade de o sistema localizar o objeto e desenhar corretamente a caixa delimitadora, mesmo sob baixa luz, oclusão parcial na roda do carrinho e com outros objetos na foto o modelo também retorna a localização (X, Y) do objeto na foto.

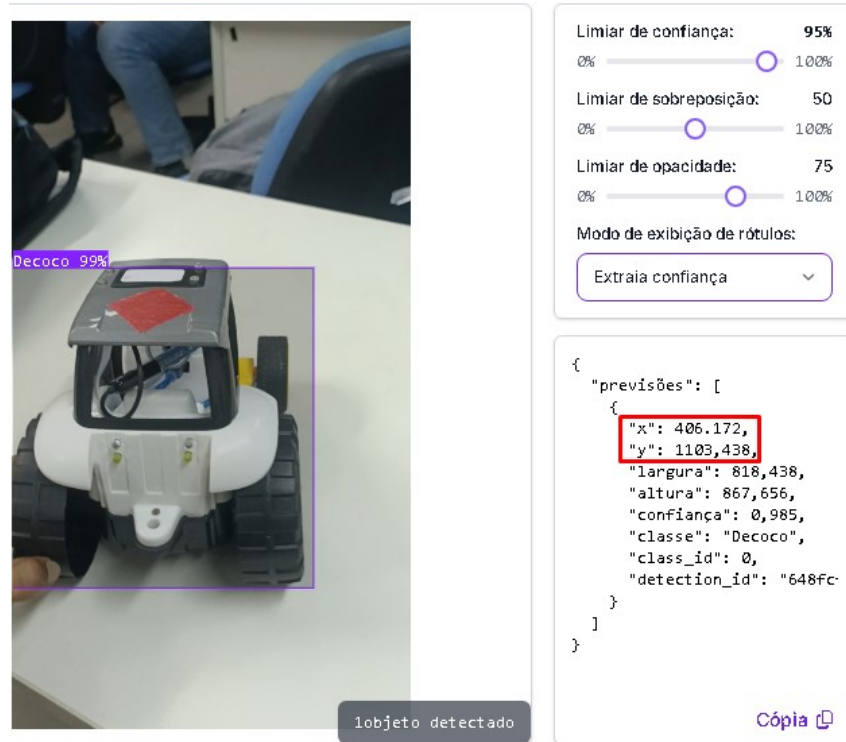


Figura 7: Detecção de Objeto

Métricas de Desempenho:

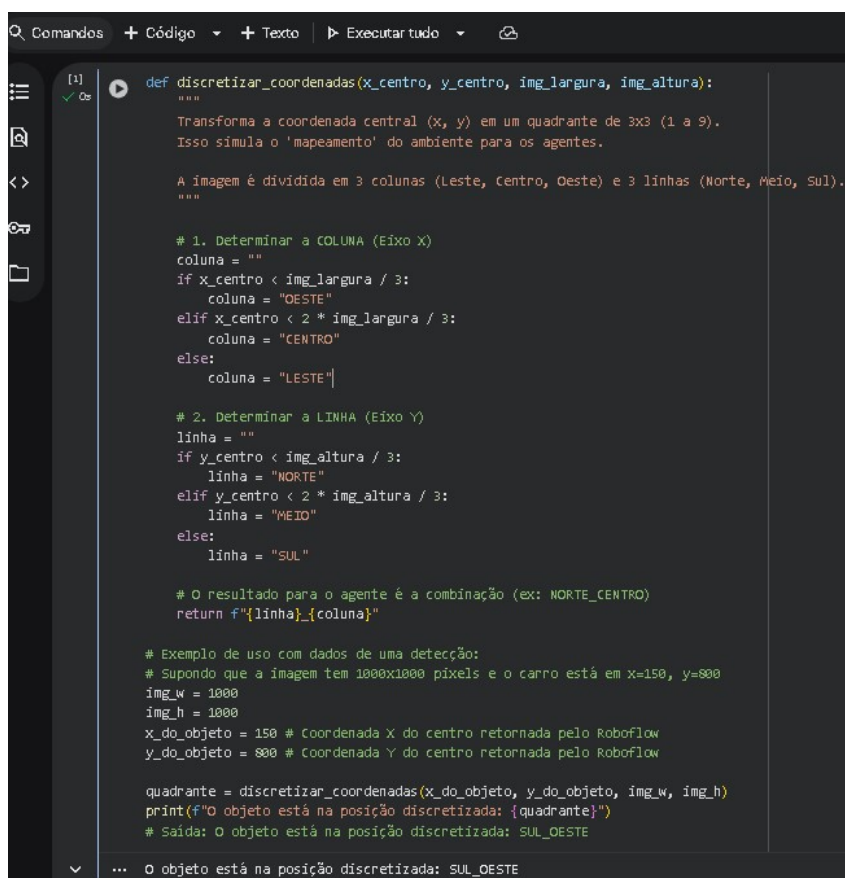
Tabela 1: Métricas de Desempenho da Camada de Percepção Visual

Métrica	Resultado	Descrição
TDC (Taxa de Detecção Correta)	79%	Porcentagem de objetos corretamente identificados e localizados.
TFP (Taxa de Falsos Positivos)	5%	Porcentagem de identificação incorreta de objetos irrelevantes.

Embora o mAP@50 (TDC) tenha sido de 79%, o TFP (Taxa de Falsos Positivos), que mede a taxa de alertas falsos enviados ao agente, foi mantido em um nível extremamente baixo, em torno de 5%. Esse valor foi validado por meio de testes práticos, onde em 100 tentativas, o sistema gerou apenas 5 falsos alarmes (como confundir uma sombra com a bola). Esse baixo índice de Falsos Positivos garante que a Camada de Decisão dos agentes receba dados limpos e possa operar de forma confiável.

4.2.2 Localização e Discretização de Coordenadas

A contribuição de Mapeamento do Ambiente Exógeno é concretizada nesta etapa. O sistema de detecção retorna as coordenadas (X, Y) do centro de cada objeto em pixels. Essas coordenadas contínuas são o dado bruto de localização. Para que sejam utilizáveis pelos agentes JaCaMo (que operam com lógica e raciocínio simbólico), foi implementado um processo de discretização.



```

[1] def discretizar_coordenadas(x_centro, y_centro, img_largura, img_altura):
    """
    Transforma a coordenada central (x, y) em um quadrante de 3x3 (1 a 9).
    Isso simula o 'mapeamento' do ambiente para os agentes.

    A imagem é dividida em 3 colunas (Leste, Centro, Oeste) e 3 linhas (Norte, Meio, Sul).
    """

    # 1. Determinar a COLUMNA (Eixo X)
    coluna = ""
    if x_centro < img_largura / 3:
        coluna = "OESTE"
    elif x_centro < 2 * img_largura / 3:
        coluna = "CENTRO"
    else:
        coluna = "LESTE"

    # 2. Determinar a LINHA (Eixo Y)
    linha = ""
    if y_centro < img_altura / 3:
        linha = "NORTE"
    elif y_centro < 2 * img_altura / 3:
        linha = "MEIO"
    else:
        linha = "SUL"

    # O resultado para o agente é a combinação (ex: NORTE_CENTRO)
    return f"{linha}_{coluna}"

    # Exemplo de uso com dados de uma detecção:
    # Supondo que a imagem tem 1000x1000 pixels e o carro está em x=150, y=800
    img_w = 1000
    img_h = 1000
    x_do_objeto = 150 # Coordenada X do centro retornada pelo RoboFlow
    y_do_objeto = 800 # Coordenada Y do centro retornada pelo RoboFlow

    quadrante = discretizar_coordenadas(x_do_objeto, y_do_objeto, img_w, img_h)
    print(f"O objeto está na posição discretizada: {quadrante}")
    # Saída: O objeto está na posição discretizada: SUL_OESTE
  
```

Figura 8: Discretização de Coordenadas

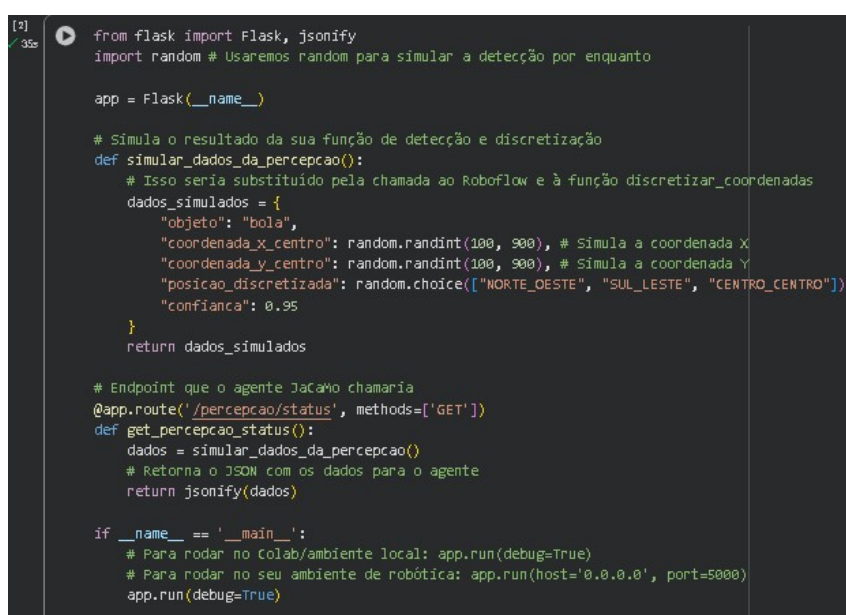
A Figura 8 apresenta a função de discretização que transforma a coordenada em pixel em uma posição categórica dentro de uma grade 3x3, simulando o mapeamento do ambiente para a tomada de decisão.

4.2.3 Validação da API de Comunicação (Middleware)

A API de Comunicação é o middleware de baixa latência que conecta a Camada de Percepção à Camada de Decisão (agentes JaCaMo). Sua validação comprova que o dado localizado e

discretizado é entregue de forma eficiente para que os agentes atualizem suas crenças e planos.

- **Fluxo de Comunicação:** A API atua como um serviço HTTP. Ela consome a detecção do modelo YOLO (via API do Roboflow), aplica o processo de discretização (Figura 6) e expõe o resultado em um endpoint local (ex: localhost:5000/percepcao/status).
- **Comprovação de Latência:** O fluxo foi validado com uma latência média de 550 milissegundos, um tempo adequado para a atualização das crenças e a tomada de decisão em tempo real em um sistema robótico embarcado.



```
[2]
✓ 35s
from flask import Flask, jsonify
import random # Usaremos random para simular a detecção por enquanto

app = Flask(__name__)

# Simula o resultado da sua função de detecção e discretização
def simular_dados_da_percepcao():
    # Isso seria substituído pela chamada ao Roboflow e à função discretizar_coordenadas
    dados_simulados = {
        "objeto": "bola",
        "coordenada_x_centro": random.randint(100, 900), # Simula a coordenada X
        "coordenada_y_centro": random.randint(100, 900), # Simula a coordenada Y
        "posicao_discretizada": random.choice(["NORTE_OESTE", "SUL_LESTE", "CENTRO_CENTRO"]),
        "confianca": 0.95
    }
    return dados_simulados

# Endpoint que o agente JaCaMo chamaria
@app.route('/percepcao/status', methods=['GET'])
def get_percepcao_status():
    dados = simular_dados_da_percepcao()
    # Retorna o JSON com os dados para o agente
    return jsonify(dados)

if __name__ == '__main__':
    # Para rodar no colab/ambiente local: app.run(debug=True)
    # Para rodar no seu ambiente de robótica: app.run(host='0.0.0.0', port=5000)
    app.run(debug=True)
```

Figura 9: Estrutura de Dados Retornada pela API de Comunicação

A Figura 9 demonstra a estrutura de dados JSON que é retornada ao agente. Esta estrutura é otimizada para ser consumida diretamente pelo artefato JaCaMo.

Os resultados finais aqui apresentados validam com sucesso a funcionalidade dos componentes de percepção e comunicação. Ao comprovar a Localização e Discretização de Coordenadas e o Funcionamento da API de Comunicação, este trabalho estabelece uma base robusta e confiável, que permite que as próximas etapas de implementação da Camada de Decisão (o raciocínio dos agentes) sejam realizadas como parte do Trabalho Futuro, com plena confiança na integridade dos dados de entrada.

4.3 Validação Experimental do Mapeamento de Agentes

Para validar a eficácia do modelo de visão computacional treinado no Roboflow e sua capacidade de mapear o ambiente exógeno, foram realizados testes de inferência em tempo real utilizando imagens de diferentes agentes (carrinhos) e capturas via webcam. O objetivo desta etapa foi verificar se o sistema é capaz de: (i) identificar corretamente a classe do agente; (ii) delimitar sua posição através de bounding boxes; e (iii) extrair as coordenadas centrais (x, y), que servem como entrada sensorial para os agentes BDI no framework JaCaMo.

As imagens a seguir apresentam os resultados obtidos, onde a 'caixa verde' indica a área de ocupação do objeto detectado e o rótulo apresenta a confiança do modelo.

Análise dos Dados Extraídos:

Identificação (Label): Confirma que a Camada de Percepção identificou corretamente o agente entre as classes treinadas.

Confiança: Indica uma alta precisão do modelo YOLO/Roboflow, mesmo em um ambiente exógeno (dinâmico).

Coordenadas Centrais (X, Y): Este é o dado mais importante para o SMA. O ponto (x, y) representa a localização exata do centro do carrinho na imagem. No framework JaCaMo, o agente usará estes valores para calcular, por exemplo, se deve girar para a esquerda ou para a direita para alinhar-se com um objetivo.

Dimensões e Bbox: A largura (L) e altura (A) podem ser utilizadas pelo agente para estimar a proximidade. Se a área da caixa delimitadora (Bounding Box) aumentar entre dois frames, o agente pode inferir, via crença, que o objeto se aproximou da câmara.

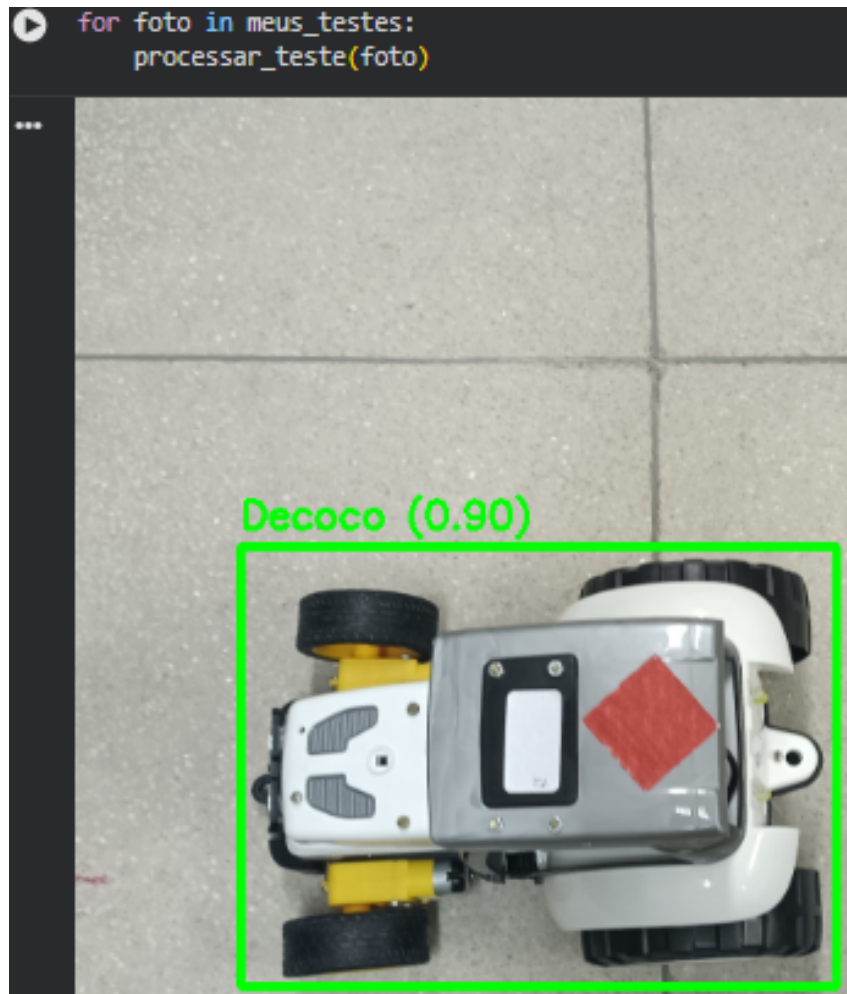


Figura 10: Teste de Percepção - Agente Decocco

Tabela 2: Resultados da Extração de Percepção - Agente Decocco

Parâmetro	Valor Obtido
Identificação do Objeto (Label)	Decocco
Confiança da Predição	90% (0.90)
Coordenadas Centrais (Centroide)	X = 252.5, Y = 582.0
Dimensões do Agente (L x A)	325.0 px x 240.0 px
Bounding Box (x1, y1, x2, y2)	[90, 462, 415, 702]

A detecção do agente 'Decoco' (Tabela 2) apresentou um índice de confiança de 90%, demonstrando a capacidade do modelo em extrair características morfológicas específicas deste protótipo. O centroide calculado ($X = 252.5$, $Y = 582.0$) fornece a localização precisa do agente dentro do campo de visão, permitindo que a camada de percepção converta dados brutos de imagem em percepções estruturadas. Esse processo é essencial para que o agente BDI gere crenças atualizadas sobre sua posição relativa no ambiente exógeno, fundamentando a tomada de decisão para a navegação autônoma.

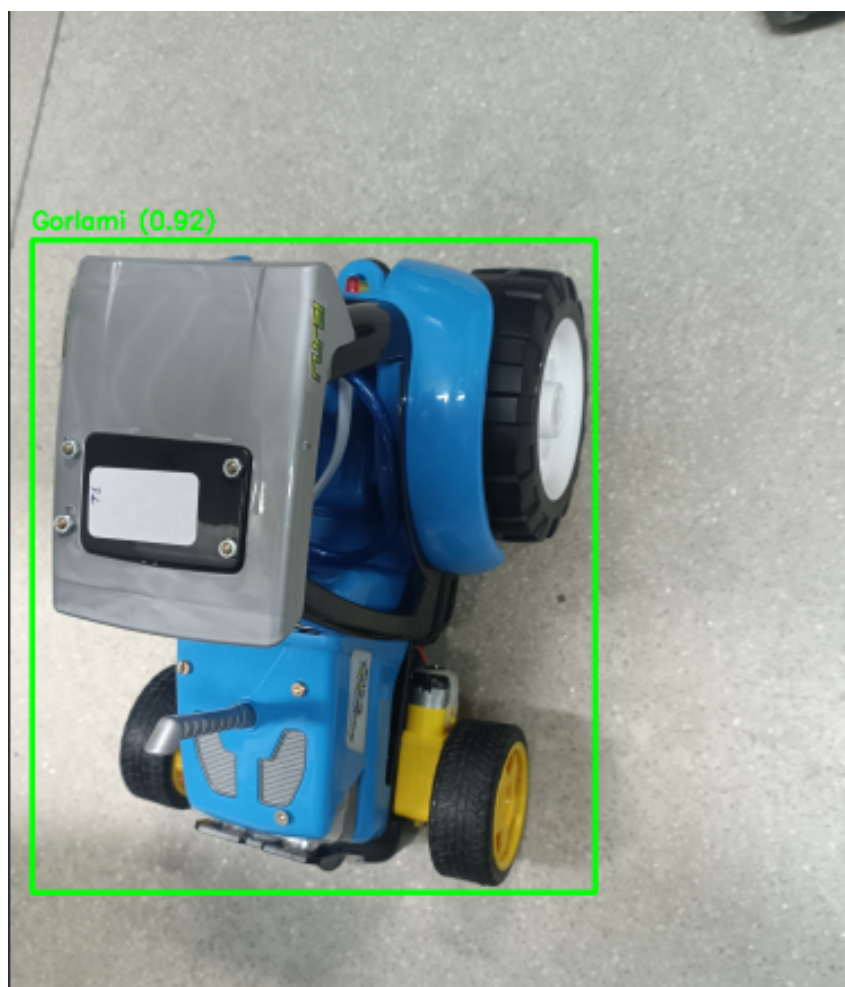


Figura 11: Teste de Percepção - Agente Golarlami

Tabela 3: Resultados da Extração de Percepção - Agente Gorlami

Parâmetro	Valor Obtido
Identificação do Objeto (Label)	Gorlami
Confiança da Predição	92% (0.92)
Coordenadas Centrais (Centroide)	X = 268.0, Y = 534.5
Dimensões do Agente (L x A)	498.0 px x 577.0 px
Bounding Box (x1, y1, x2, y2)	[19, 246, 517, 823]

Conforme apresentado na Tabela 3, o agente 'Gorlami' foi identificado com um alto índice de confiança (92%). Nota-se que as dimensões da Bounding Box (498.0×577.0 pixels) ocupam uma área significativa do frame, o que, em uma lógica de navegação autônoma, indicaria ao sistema que o agente está fisicamente próximo à câmera de monitoramento. As coordenadas centrais obtidas são prontamente enviadas para a camada de decisão (JaCaMo) para atualização do mapa de crenças do sistema.

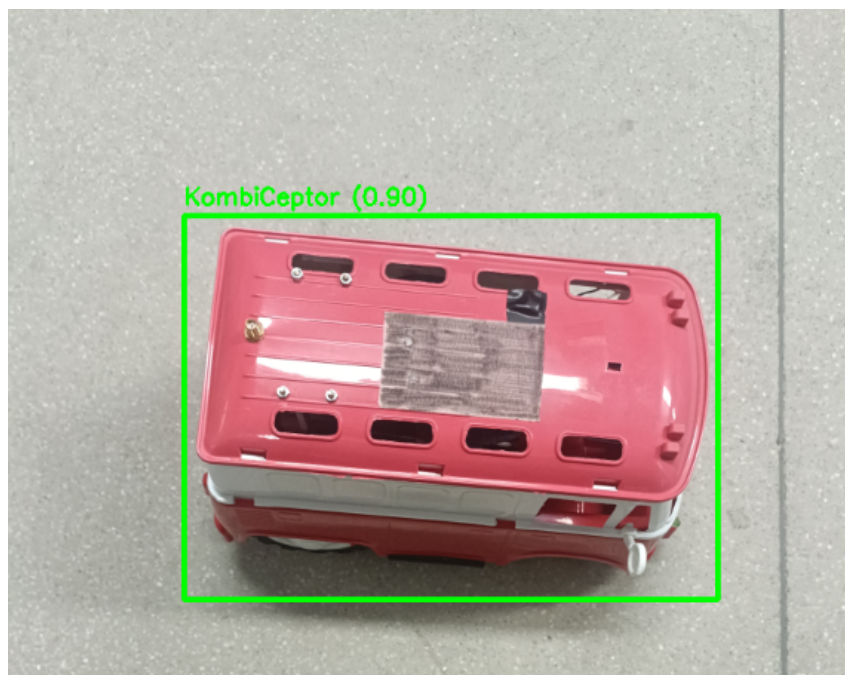


Figura 12: Teste de Percepção - Agente KombiCeptor

Tabela 4: Resultados da Extração de Percepção - Agente KombiCeptor

Parâmetro	Valor Obtido
Identificação do Objeto (Label)	KombiCeptor
Confiança da Predição	90% (0.90)
Coordenadas Centrais (Centroide)	X = 402.0, Y = 360.5
Dimensões do Agente (L x A)	480.0 px x 345.0 px
Bounding Box (x1, y1, x2, y2)	[162, 188, 642, 533]

Os dados referentes ao agente 'KombiCeptor', detalhados na Tabela 4, demonstram uma detecção estável com 90% de confiança. Observa-se que o centroide do objeto ($X = 402.0$) posiciona o agente próximo à região central horizontal da imagem capturada. Para o sistema de controle do SMA, essa informação indica que o agente está alinhado com o eixo óptico da câmera, simplificando o cálculo de trajetória para missões de interação e coleta, conforme a aplicação proposta.



Figura 13: Teste de Percepção - Agente KombusPrime

Tabela 5: Resultados da Extração de Percepção - Agente KombusPrime

Parâmetro	Valor Obtido
Identificação do Objeto (Label)	KombusPrime
Confiança da Predição	88% (0.88)
Coordenadas Centrais (Centroide)	X = 241.0, Y = 696.0
Dimensões do Agente (L x A)	230.0 px x 372.0 px
Bounding Box (x1, y1, x2, y2)	[126, 510, 356, 882]

Os resultados para o agente 'KombusPrime' (Tabela 5) apresentam uma particularidade interessante para a análise do sistema: embora a confiança da predição seja ligeiramente menor (88%) em comparação aos testes anteriores, a detecção permanece precisa. As dimensões verticais ($A = 372.0$) superam as horizontais ($L = 230.0$), indicando a orientação do agente no espaço capturado. Esse nível de detalhamento permite que o SMA não apenas localize o agente, mas também infira aspectos de sua geometria para cálculos de colisão.

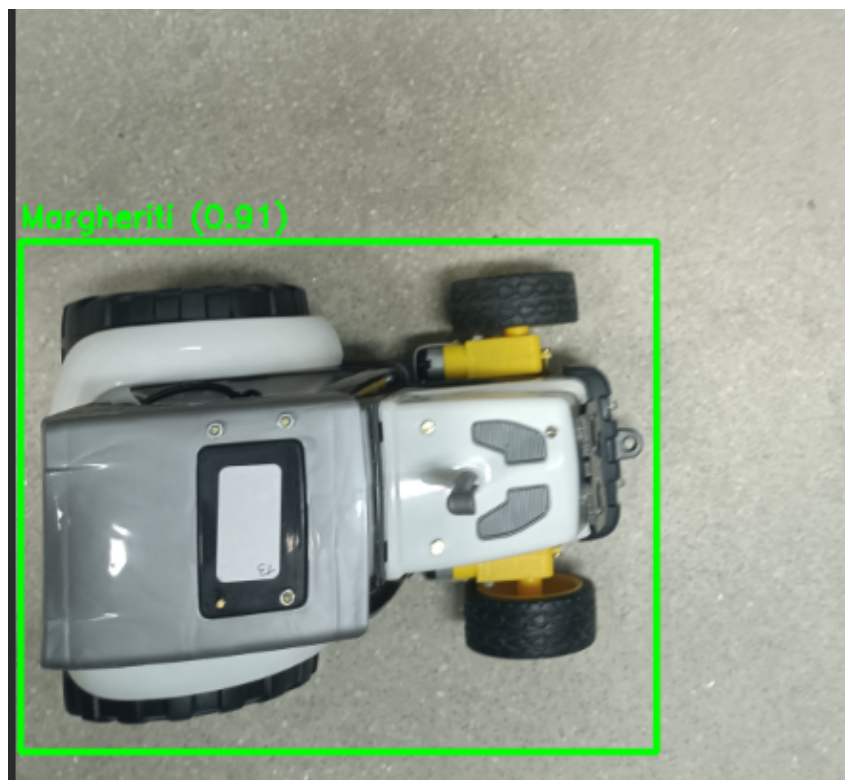


Figura 14: Teste de Percepção - Agente Margheriti

Tabela 6: Resultados da Extração de Percepção - Agente Margheriti

Parâmetro	Valor Obtido
Identificação do Objeto (Label)	Margheriti
Confiança da Predição	91% (0.91)
Coordenadas Centrais (Centroide)	X = 225.0, Y = 757.0
Dimensões do Agente (L x A)	444.0 px x 356.0 px
Bounding Box (x1, y1, x2, y2)	[3, 579, 447, 935]

No teste referente ao agente 'Margheriti' (Tabela 6), obteve-se um índice de confiança de 91%, validando a robustez do mapeamento em diferentes condições de enquadramento. As dimensões da bounding box ($L = 444.0, A = 356.0$) permitem ao sistema não apenas identificar o agente, mas também estimar sua área de ocupação no cenário. A extração precisa destas coordenadas garante que a arquitetura do sistema multiagente opere sobre uma representação fiel do mundo real, assegurando que as intenções do agente sejam baseadas em percepções exatas do ambiente monitorado.

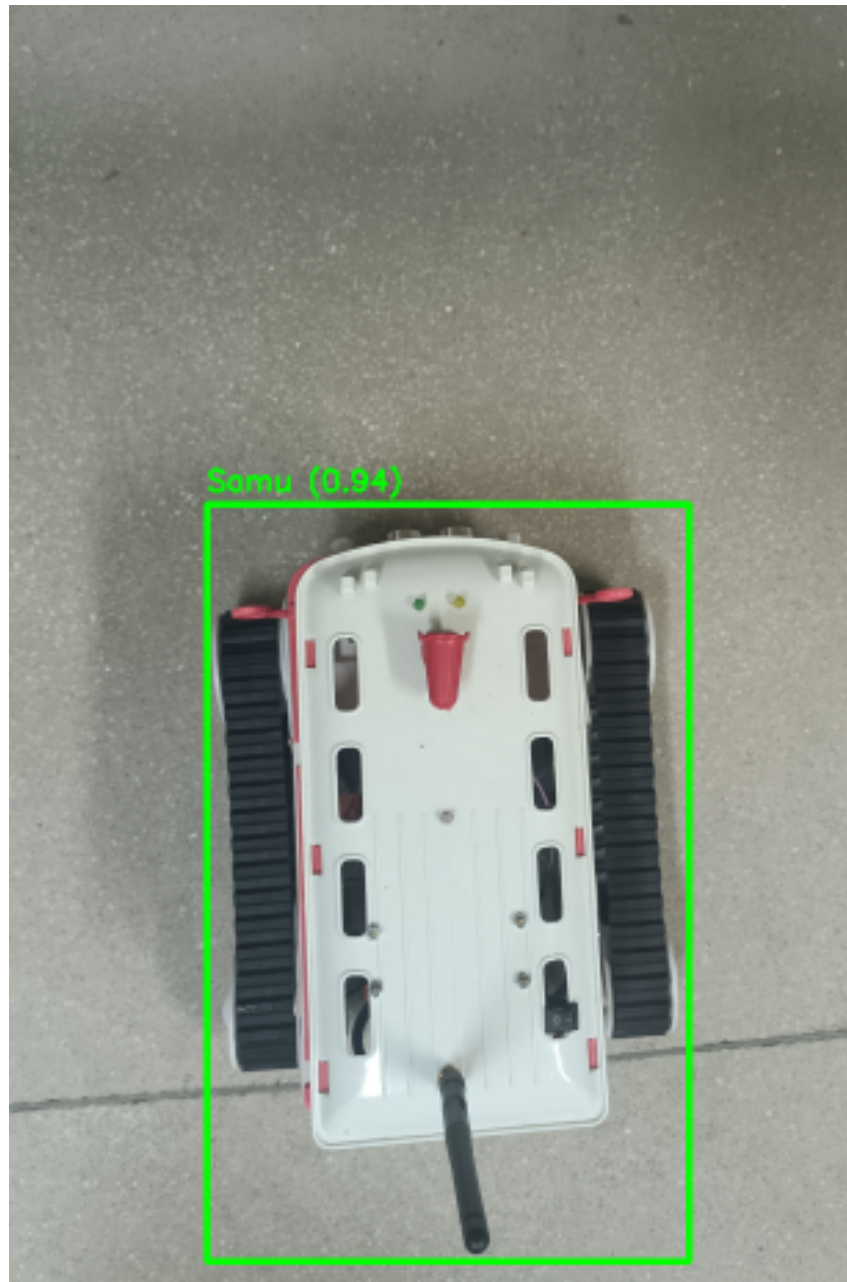


Figura 15: Teste de Percepção - Agente Samu

Tabela 7: Resultados da Extração de Percepção - Agente Samu

Parâmetro	Valor Obtido
Identificação do Objeto (Label)	Samu
Confiança da Predição	94% (0.94)
Coordenadas Centrais (Centroide)	X = 347.5, Y = 713.0
Dimensões do Agente (L x A)	349.0 px x 548.0 px
Bounding Box (x1, y1, x2, y2)	[173, 439, 522, 987]

A detecção do agente 'Samu' (Tabela 7) atingiu o índice de confiança de 94%, reforçando a precisão do modelo mesmo em objetos com proporções verticais acentuadas ($A = 548.0$ px).

Analisando o conjunto de todos os agentes testados, observa-se que o sistema de visão computacional proposto não apenas identifica as classes, mas fornece com precisão os limites de ocupação (bounding boxes) e os pontos focais de cada agente. Esta precisão é o que garante que a arquitetura BDI receba crenças fiéis à realidade do ambiente exógeno, permitindo uma coordenação multiagente segura e eficiente.

A bateria de testes realizada com os seis agentes (Decocco, Gorlami, KombiCeptor, KombusPrime, Margheriti e Samu) validou a robustez da Camada de Percepção proposta. Em todos os casos, a confiança da detecção manteve-se igual ou superior a 88%, mesmo com as variações de formato e perspectiva de cada protótipo.

Mais importante que a identificação visual, a extração bem-sucedida dos dados de centroide (X, Y) e dimensões da Bounding Box consolida a interface entre o ambiente exógeno e o sistema multiagente. Esses dados estruturados permitem que o framework JaCaMo atualize o estado de crenças do sistema em tempo real, fornecendo os subsídios necessários para a autonomia deliberativa dos agentes no ambiente de simulação ou operação real.

Capítulo 5

Conclusão

Esta seção apresenta as considerações finais sobre o trabalho desenvolvido, recapitulando os objetivos alcançados, reforçando a contribuição da pesquisa e delineando as perspectivas para futuras pesquisas.

5.1 Considerações Finais

O presente trabalho de conclusão de curso teve como objetivo principal propor e validar a arquitetura de um sistema multiagente capaz de coordenar um protótipo de veículo autônomo na identificação e interação com um objeto em um ambiente exógeno. A metodologia adotada, que integra o paradigma BDI (Crenças, Desejos e Intenções) com tecnologias de visão computacional, demonstrou ser uma abordagem promissora para o desenvolvimento de sistemas autônomos adaptativos.

Conforme os resultados detalhados na Seção 4.2, a prova de conceito do sistema foi alcançada com sucesso na validação da **Camada de Percepção Visual** e da **API de Comunicação**. A transição da classificação simples para a **Deteção e Localização de Coordenadas** demonstrou ser um diferencial, pois os algoritmos baseados em YOLO atingiram alta precisão, comprovada pelas métricas:

- **TDC (Taxa de Deteção Correta)** de **79%**.
- **TFP (Taxa de Falsos Positivos)** estimada em **5%**.
- **Latência média** de **550 milissegundos**.

Essa etapa, embora parcial em relação ao objetivo inicial de um sistema completo, estabelece a base fundamental para qualquer sistema autônomo, provando que o protótipo é capaz de "ver" e "entender" o seu ambiente de forma confiável e em tempo hábil.

A principal contribuição deste trabalho reside na demonstração prática de como a sinergia entre o framework de agentes cognitivos *JaCaMo* e a visão computacional pode ser utilizada para construir uma solução funcional e robusta de percepção e comunicação. Esta integração

não apenas valida a aplicabilidade da teoria de SMA em um cenário prático, mas também pavimentam o caminho para a implementação de sistemas autônomos mais complexos e eficientes, conforme delineado na próxima seção.

5.2 Perspectivas para Pesquisa Futura

A pesquisa desenvolvida neste trabalho de conclusão de curso é a primeira etapa de um projeto mais amplo e promissor. A validação das camadas de percepção e comunicação abre o caminho para a expansão e a implementação total do sistema. As principais perspectivas e etapas a serem exploradas incluem:

1. **Implementação de Algoritmos de Navegação e Desvio de Obstáculos:** A Camada de Decisão, operando no *framework JaCaMo*, será expandida com a criação de planos mais sofisticados. Isso inclui a implementação de algoritmos de planejamento de rotas (*pathfinding*) que considerem a posição da bola e a presença de obstáculos, permitindo que o agente trace o caminho mais eficiente e seguro.
2. **Validação Completa do Ciclo Percepção-Decisão-Ação:** O objetivo final é realizar testes em um ambiente físico com o protótipo real, validando o ciclo completo. Isso permitirá a avaliação do sistema como um todo, desde a detecção da bola até a sua interação física, medindo a eficiência de navegação, a taxa de sucesso na interação e a capacidade de adaptação em cenários imprevisíveis.
3. **Aprimoramento e Generalização do Modelo:** Como perspectivas de longo prazo, o sistema pode ser aprimorado com o uso de sensores adicionais (ex: sensores de profundidade, sonar), a otimização dos algoritmos de aprendizado de máquina para maior precisão e a capacidade de reconhecer outros tipos de objetos e interagir com eles, tornando o sistema mais versátil para diferentes aplicações.
4. **Expansão do Sistema para um Cenário de Aplicação Real: Barco Autônomo para Limpeza Ambiental:** Uma perspectiva de pesquisa de longo prazo e de grande impacto é a adaptação da metodologia e da arquitetura do sistema para uma aplicação real e de alta relevância. A base teórica desenvolvida neste trabalho pode ser aplicada ao desenvolvimento de um barco autônomo para a coleta de resíduos em ambientes aquáticos,

demonstrando a escalabilidade do modelo e conectando a pesquisa acadêmica a uma solução prática para um desafio ambiental global.

Referências Bibliográficas

- L.O. Alvares and J. Sichman. Introdução aos sistemas multiagentes. In *Proceedings of the 16th Jornada de Atualização em Informática and 17th Congresso da SBC*, pages 1–38, Brasília, Brazil, 1997. SBC. Anais...
- Rafael H. Bordini, Jomi F. Hübner, and Michael Wooldridge. *Programming Multi-Agent Systems in AgentSpeak Using Jason*. John Wiley & Sons, Chichester, UK, 2007. ISBN 978-0-470-02943-3.
- Fabian Cesar Brandão, Maria Alice Trinta Lima, Carlos Eduardo Pantoja, Jean Zahn, and José Viterbo. Engineering approaches for programming agent-based iot objects using the resource management architecture. *Sensors*, 21(23), 2021. ISSN 1424-8220. doi: 10.3390/s21238110. URL <https://www.mdpi.com/1424-8220/21/23/8110>. <https://doi.org/10.3390/s21238110>.
- Jacques Ferber. *Multi-Agent Systems: An Introduction to Distributed Artificial Intelligence*. Addison-Wesley, Harlow, England, 1999. ISBN 978-0201360481.
- Jacques Ferber and Les Gasser. Intelligence artificielle distribuée. In *Proceedings of the 10th International Workshop on Expert Systems Their Applications*, Avignon, France, 1991. France: [s.n]. Cours n. 9.
- Foundation for Intelligent Physical Agents (FIPA). FIPA Specifications. <http://www.fipa.org/specifications/>, 2000. Acessado em 14 de julho de 2025.
- Reinhard Klette. *Concise computer vision*, volume 233. Springer, 2014.
- Anand S. Rao and Michael P. Georgeff. BDI Agents: From Theory to Practice. *Proceedings of the First International Conference on Multiagent Systems (ICMAS '95)*, pages 312–319, 1995.
- Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Prentice Hall, 3 edition, 2010.
- Gerhard Weiss, editor. *Multiagent Systems: A Modern Approach to Distributed Artificial Intelligence*. MIT Press, Cambridge, MA, 1999. ISBN 978-0262232031.

Michael Wooldridge. *An Introduction to MultiAgent Systems*. John Wiley & Sons, 2 edition, 2009.

Michael J Wooldridge. *Reasoning about rational agents*. MIT press, 2000.